善解：結合神經重排的開源注音輸入法

Nanako Tsai*

版本 v1，2026-11-12

本文採創用 CC 姓名標示 4.0 國際授權條款（CC BY 4.0）。
https://creativecommons.org/licenses/by/4.0/

摘要

注音輸入打出來的是一串音節，輸入法要在所有同音的字串裡，選出使用者想打的那一串。本文介紹開源的 macOS 注音輸入法「善解」（以 2026-11-11 發佈的 v0.7.0 為準），並回答一個問題：它的選字是靠什麼變準的。系統是一個分層的 n-gram 解碼器，後面接一個 1,465 萬參數的字元語言模型，它只能在分差閘門之後重排解碼器前 8 名的路徑。主要的發現是：進步來自語域對的訓練文字，以及重排模型的分數怎麼和解碼器的分數融合，不是來自更大的神經部件。最大的一步是用論壇與聊天頻道文字重建的那一代模型；更大的重排模型、長前文、listwise 與蒸餾目標、大型語言模型判斷器都沒過事先寫好的標準；同一份配方重訓的兩個重排模型，在一份聊天集上互換 65–100 列、淨值在 ±12 以內，多數模型改動沒有超過這個雜訊底線。部件用配對統計判讀；確認從 2026-10-18 起要預先登記，在沒看過的資料上只跑一次。一共跑了 13 次單次確認，8 次成立、5 次不成立。也有改動沒有經過單次確認就出貨，例如最大的那一步、較早的一個讀音項、詞包，以及 v0.7.0 幾條較小的規則；這幾類裡紀錄有數字的，判讀用的是檢查集與已經看過的集合。在作者自己 4,958 個子句的聊天開發集（選值時看過）上，聊天設定的第一名正確率從 v0.4.0 的 84.89% 升到 v0.7.0 的 94.07%（書面設定下從 83.56% 升到 93.71%）。在發版監測用的保留集上，v0.5.0→v0.6.0 讓公開口語半邊從 115 列對升到 124 列對（共 150 列；精確 McNemar $p = 0.012$；這是那個半邊第五次回報數字），新啟用的新聞半邊從 125 升到 128（$p = 0.25$）；v0.6.1→v0.7.0 之後是 127（$p = 0.25$）與 126（$p = 0.63$）。所有證據都來自一位使用者的聊天語域與工具產生的讀音。

1 引言

台灣使用者用注音打國語時，每一組按鍵是一個帶聲調的音節，而幾乎每個音節都對應許多字。輸入法因此在一個硬限制下解一個序列標註問題：輸出字串的讀音必須剛好是打進來的音節，而系統要盡量選出使用者想打的那一串，每一鍵只有幾毫秒。有兩件事讓它比聽寫式的轉換更難。第一，同一個使用者寫兩種語域——隨意的聊天和書面文字——用詞與詞頻差到一組參數兩邊都顧不好。第二，使用者在意的錯集中在語言本身局部分不出來的同音對（在／再、的／得、做／作）、專有名詞，以及編輯過的文字裡沒有的口語。

善解是一個開源（Apache-2.0）的 macOS 輸入法，只有一個明說的優先順序：送進 App 的文字永遠是本機詞庫 lattice 產生的候選之一，而在候選之間的選擇要在資料允許的範圍內盡量正確 [35]。本文描述 v0.7.0（2026-11-11 發佈）出貨的系統，以及開發約六週累積的證據。本文問三個問題。

問題一 一個已經有 n-gram 解碼器的注音輸入法，第一名正確率靠什麼往上：更多使用者語域的訓練文字、更多解碼器結構，還是更大的神經重排模型？

*GitHub：https://github.com/Nanako0129/shanjie；網站：https://shanjie.nyanako.com。

問題二 小型神經重排模型要照什麼決策規則覆寫解碼器的第一名，才不會改壞的比改對的多？

問題三 在開發資料上選出來的結果能信到什麼程度：到沒看過的資料上會縮多少？判斷一個模型改動時要對照的重訓雜訊有多大？

我們的發現，以及各自依據的證據：

1. **語域對的文字，不是更大的神經模型**（問題一）。聊天開發集上最大的一步（4,958 個子句裡 4,358→4,645 對）是 bigram 與重排模型都用論壇與聊天頻道文字重建的那一代；重建的 bigram 單獨讓這個集合從 4,366 到 4,534，用同樣文字重訓的重排模型再加 $+88/-26$（表 5）。更大的重排模型、長前文、listwise 目標、從更強的教師蒸餾、大型語言模型判斷器，在測過的條件下各自都沒過事先寫好的標準（表 6）。
2. **重排模型的證據怎麼融合，和它的權重一樣重要**（問題二）。口語表當成解碼器的一項時，整條流程跑完改壞太多列；同一張表改放在融合裡、過軟閘門，確認成立。最後一個重排模型在書面設定的進步幾乎全來自融合格，不是權重。起作用的不是硬性的分差門檻：在標準檔拿掉它，2,992 列公開聊天一列都不變，2,182 列書面只變 2 列，4,919 列私人聊天只變 1 列（表 4；第 5.3、5.4 節；表 1 第 3、12、13 列）。
3. **重訓的流動就是雜訊底線**（問題三）。同一份配方訓出來的兩個重排模型，在一份聊天集上互換 65–100 列、淨值在 ± 12 以內；接續訓練把互換壓到 21–33 列；而出貨的融合格是替某一個種子選的（第 5.4 節）。
4. **選值用的集合偏樂觀**（問題三）。選出來的格，改壞 $\div$ 改對的比例從選值集到新資料每一次檢查都上升；13 次單次確認有 5 次不成立，其中 4 次在已經看過的資料上是過的（表 1）。
5. **一張轉換表悄悄改寫了正解與訓練文字**：127 個詞庫裡的詞，是一類使用者回報錯字的根因（第 5.4 節）。

本文的貢獻是：(i) 選字流程的描述，每一條決策規則都寫成明確的分數公式並對照程式（第 3 節）；(ii) 單人輸入法專案的評測流程，連同全部單次確認的帳，成立與不成立的都列（第 4、5 節）；(iii) 負面結果，依失敗的種類分類，寫明各自沒達到的條件（第 6 節）。專案代號、日期與每個部件的完整紀錄放在附錄。

這份工作是怎麼做的。 實作、量測與本文的文字由 LLM 程式代理完成。全文裡「agent」指一個 LLM session；「獨立的 agent」指一個新開的 session，它沒有參與被審查的那項實作，只拿到一份書面的任務說明。程式審查、驗收與保留集量測由獨立的 agent 做；設計審閱與研究顧問意見來自幾個模型。本文每一個數字與陳述由作者負責。

2 相關工作

這一節從五個面向把既有工作和本文對照：任務、輸出空間、給模型的輸入、評測，以及本文的定位。

任務。 把一串拼音按鍵轉成漢字，從 Chen and Lee [11] 與 Gao et al. [16] 的拼音研究開始就當成統計語言模型的解碼問題，之後有和打字錯誤一起處理的做法 [22]，近年則用神經模型 [20, 42, 54, 58]。實際出貨的引擎另外處理字典與語言模型的壓縮 [26]，或用有限狀態解碼器 [19]。注音方面，開源引擎有新酷音（libchewing）[44]、RIME [40]、小麥注音 [37]（善解用它的詞庫當基底層）與唯音 [53]。善解的解碼器屬於統計這一類：絕對折扣 [10, 36] 加 Kneser–Ney 接續分布 [24]（訓練計數是詞圖上的期望次數，所以接續次數用四捨五入的整數加門檻，不是 [57] 的分數次數版本；Pitman–Yor 推廣 [43] 沒有做），詞類則是對 [8] 的類別 bigram 目標做 exchange 分群（參見 [30]）。

輸出空間。　這些系統的輸出都限制在讀音符合輸入的字串；PinyinGPT [42] 把這個限制寫明，把輸出分布限制在同音字。善解更嚴：神經模型從不產生文字，只重排詞庫 lattice 上最好的 8 條路徑。rime-wanxiang 與 librime-witogram [2, 28] 報告過一個負面結果，專案把它採納為設計規則：把搭配表換成原始概似後 top-1 下降，所以善解的重排模型只有在融合分數超過一個在指定資料上選定的分差門檻時，才能覆寫 n-gram 的第一名（第 3.5 節）。這個閘門是 [17] 意義下的選擇性預測規則，和「學會交給專家」[34] 相關；我們不主張它是 [18] 意義下校準過的機率。

輸入。　各系統給神經模型看的東西不同。0c（靈犀注音）公開一個 Transformer，用前文最後 64 字加讀音重排統計引擎的候選，在它自己的 6,000 句新聞集上報統計引擎首選 76.9%、加 Transformer 86.0% [1, 15]（自報數字，讀音由字典合成、集合不同，不能和本文任何數字比；§rl-051）。pinyin-ime-reranker-4M 的模型卡 [31] 報告前綴狀態在候選與按鍵之間共用後 p95 從 20.64 ms 降到 8.61 ms；Zenzai [3] 用保留符號分段、前文放在讀音之前。善解的重排模型看到的是候選字串與最多兩個字的前文，看不到讀音。這批文獻裡有兩個想法試過、沒有採用：[41] 與 [45] 啟發的判別式（listwise）目標，以及交叉熵差的資料挑選 [33]（第 6 節）。Kuhn 與 De Mori 的快取語言模型 [27] 是規劃中逐次累加自學的參照。

評測。　KySS [21] 與 Jiang et al. [23] 對適應式中文輸入法的強健性分析，促成專案重播工具的按鍵分數；AJIMEE-Bench [4] 促成判讀集凍結時切成固定子集。配對檢定照 [12]（精確 McNemar）與 [25]（配對 bootstrap）；自然語言處理裡顯著性檢定的做法見 [5, 13]。機器學習文獻裡有兩條線對應本文說的改對、改壞與流動：模型更新造成的退步與預測流動 [32, 55, 56]，以及種子等來源替基準分數帶來的變異 [7, 39]。本文的分層集合（判讀集只用一次、保留集計次）是對 [6, 14, 38] 研究的「反覆使用同一份資料」問題的人工解法；在讀數字之前先 commit 主判讀，就是 [52] 說的預先登記。

本文的定位。　任務是帶聲調的注音轉成台灣寫法的繁體中文；輸出空間是本機詞庫 lattice 上的路徑；輸入是打進來的音節加最多兩個字的前文；評測是對出貨版本的配對比較，加單次確認。本文不提出新模型，而是就一個開源系統報告哪些改動有用、哪些沒用、量測本身有多可靠。專案第一天對三套既有輸入法的單日快照與它的限制在附錄 F；改過設計決定的文獻列在 §ip-008。

3　選字流程

圖 1 是從按鍵到畫面的流程；下面各小節定義每一格。

這一節寫的是 v0.7.0 出貨的流程。前一版 v0.6.1 和這裡的描述有三處較大的不同：重排模型的權重與融合格是前一組；讀音項放在重排的融合裡、不在解碼器裡（附錄 A.7）；沒有口語項句首的那個估法（第 3.6 節），也沒有句首單字的學習規則（附錄 A.8）。重排模型的權重、融合格、解碼器裡的讀音項與學習規則各有一次單次確認（表 1 第 11、12–13、6、9 列）；口語項句首的那個估法沒有自己的單次確認：第 11 列判讀的是書面設定，這一項在書面設定不作用；第 12、13 列聊天設定的判讀裡，兩臂都帶著它（§rl-110、§rl-113、§rl-114）。v0.7.0 另外加的三件較小的事也沒有單次確認，在描述它們的地方標明：讀音項裡同分讀音的規則（附錄 A.7）、重排遮罩裡五個詞的保護詞表（第 3.5 節）、第三個預設開的詞包（附錄 A）。v0.7.0 的發版數字是表 2 與表 3 的最後一列，表 4 把它的系統逐項拆開。

3.1　架構與限制

引擎是 C ABI 後面的 Rust 核心，由 Swift InputMethodKit 殼驅動。核心用讀音符合所打音節的詞庫條目建 lattice，用 n-gram 模型解碼，把最佳路徑交給殼；殼顯示在組字區，並把一個非同步的重排工作排進 utility 佇列。重排器只能重排解碼器的前 8 條路徑，從不產生文字。前文從

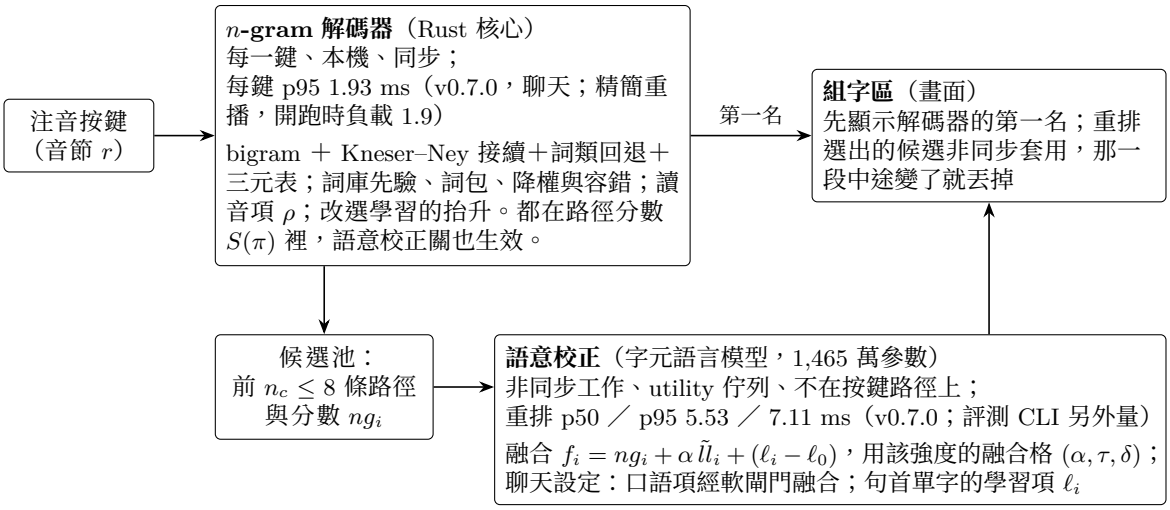


圖 1: 選字流程。解碼器每一鍵都跑，第一名立刻顯示；重排器只在背景重排候選池、從不產生文字。解碼器那一格裡的項在兩種設定、三檔、語意校正關都生效；口語項與強度的三元組 (α, τ, δ) 只在重排裡作用。每鍵的數字是同步按鍵路徑的精簡重播（只載解碼器的模型檔；不載詞包、沒有前文、沒有語意校正），不是完整 App 的延遲（第 3.8 節）。

client App 讀（最多 256 個 UTF-16 單位，在最後一個換行處截斷，上限 64 個 grapheme，同一則訊息、跨標點）；解碼器與學習存檔只用它結尾的兩個漢字，重排器用模型檔宣告的長度（每一個出貨模型都是兩個字）。殼依前景 App 的 bundle identifier 選聊天或書面設定；使用者可以全域或逐 App 釘住一種設定。

專案一開始定下的驗收目標（§plan-003）：保留集的候選涵蓋（oracle@64）$\ge 98\%$、每鍵 p95 延遲 < 16 ms、重排 p95 < 300 ms 且非同步，以及一次改選的學習不得污染其他句子。

詞庫、符號與學習。 詞庫、從它導出的先驗 $lp(w)$ 與扣分 $\eta(w)$、改選學習的規則，以及全部符號的對照表，放在附錄 A。正文只需要三件事：每個詞庫條目 w 是一個（讀音，字串）對，帶一個 $\log_{10}$ 先驗 $lp(w)$；學習之後的先驗是 $lp'(w)$，不會低於 $lp(w)$；$\eta(w) \ge 0$ 收集降權與容錯的扣分。

3.2 解碼目標

令 $r = r_1 \cdots r_n$ 是打進來的音節（各帶聲調）。一條路徑 $\pi = (w_1, \ldots, w_m)$ 是一串詞庫條目，讀音接起來剛好是 r；條目 w_j 覆蓋 $l_j \ge 1$ 個音節，$\sum_j l_j = n$。$\Pi(r)$ 是所有這樣的路徑；$y(\pi)$ 是字串 $w_1 \cdots w_m$。每個條目帶一個依讀音而定的詞庫先驗 $lp(w)$（$\log_{10}$；附錄 A.3）。設定 $p \in \{\text{聊天}, \text{書面}\}$ 決定插值權重 λ_p（$\lambda_{\text{聊天}} = 0.5$、$\lambda_{\text{書面}} = 0.7$）、Kneser–Ney 權重 β_p 與用的三元表 T_p。解碼器要找分數最高的路徑：

$$\hat{\pi} \approx \arg\max_{\pi \in \Pi(r)} S(\pi), \qquad S(\pi) = \sum_{j=1}^{m} \Big[\lambda_p \log_{10} P(w_j \mid h_j) + (1-\lambda_p)\, lp'(w_j) - \eta(w_j) + \rho(w_j)\Big] + E(\pi), \tag{1}$$

其中 h_j 是 w_j 的歷史（前一個詞 v；有三元表時再加前前一個詞 u；h_1 是這一段左邊的固定詞或 $\langle s \rangle$）；lp' 是學習之後的先驗（附錄 A.8）；$\eta(w_j) \ge 0$ 收集降權與容錯的扣分、$\rho(w_j)$ 是讀音項（附錄 A.3、A.7）；$E(\pi)$ 是收尾項：這一段在句尾時是 $\lambda_p \log_{10} P(\langle /s \rangle \mid h_{m+1})$，右邊有使用者固定的詞 w_{next} 時是那個詞完整的詞項 $\lambda_p \log_{10} P(w_{\text{next}} \mid h_{m+1}) + (1-\lambda_p)\, lp(w_{\text{next}}) - \eta(w_{\text{next}})$。這個最大化是近似的：搜尋是沿音節位置的 beam，所以 $\hat{\pi}$ 是找到的最好路徑，不保證是真正的最大值。

每個位置對每個鍵（字串，最後一個詞）留一條假設，有三元表時鍵再加前一個詞，留分數最高的 $B = 64$ 條。每個讀音只試它最好的 12 個條目（`PER_KEY`），加上使用者在目前前文下學過、排在更後面的條目。分數全是 $\log_{10}$，浮點運算順序寫死：Rust 核心在 golden 檔上的輸出和 Python 參考實作逐位元組相同，候選分數相差在 10^{-9} 以內（§plan-009）。

3.3 絕對折扣的 bigram

bigram 模型對每個有保留條目的前文詞 v 存：每個保留後接詞的期望次數 $c(v, w)$（四捨五入；期望是因為訓練句在詞圖上計數）、剪枝前的總次數 $t(v)$，以及折扣 D（出貨模型 0.75）。對模型詞彙內的 w，

$$P(w \mid v) = \begin{cases} \dfrac{c(v,w) - D}{t(v)} + \mathrm{back}(v)\, pb(w) & c(v,w) > 0, \\ \mathrm{back}(v)\, q(w \mid v) & \text{其他}, \end{cases} \qquad \mathrm{back}(v) = 1 - \sum_{w' :\, c(v,w') > 0} \frac{c(v,w') - D}{t(v)}, \tag{2}$$

沒有條目的前文 $\mathrm{back}(v) = 1$。$pb(w)$ 是 w 的 unigram 回退值，沒有側檔時是 $10^{lp(w)}$（回退分布是詞庫先驗，不是語料 unigram）；$q(w \mid v)$ 是附錄 A.5 的詞類平滑回退。加權次數低於剪枝門檻（出貨的模型是 6）的二元組不留，但 $t(v)$ 用剪枝前的總次數，所以剪枝把質量移進 $\mathrm{back}(v)$。

3.4 回退的修改、三元表與讀音項

有三個檔修改這個 bigram，另有一項加進路徑分數。公式在附錄 A，這裡只用文字說明：

- *Kneser–Ney* 接續值（式 (8)）。回退值 $pb(w)$ 換成兩者的混合，權重 β_p（聊天 1、書面 0.75）：w 的不同前文個數（正規化後）與詞庫先驗。
- 詞類回退（式 (9)）。bigram 沒有保留條目時，回退值再和「類別 bigram 乘類別發射」以權重 $\mu = 0.2$ 內插；類別是 $K = 512$ 類的 exchange 分群。這一項從不碰保留的 bigram。
- 三元表（式 (10)）。前文 (u, v) 有存時，用絕對折扣的三元、回退到 bigram，並以 bigram 分數的 $F = 0.5$ 倍為下限。表（書面 7,700,964 條、聊天 950,000 條）以唯讀記憶體映射讀。
- 讀音項 $\rho(w)$（式 (11)）。單字條目用的讀音不是那個字分數最高的讀音時，扣 $\lambda_r = 2.0$ 乘上 $\log_{10}$ 分數差（分數差最多算到 4）；變調讀音、以及詞庫把辭典第一讀音排在第二的 12 個字不扣；落在詞庫底分的讀音改用多字詞重估，v0.7.0 起詞庫給同分的讀音也是。這一項在路徑分數裡、beam 照它剪枝，兩種設定、三檔、語意校正關都生效。

是分數，不是機率。 下面寫成 $P(\cdot \mid \cdot)$ 的量都是分數。式 (2) 的 bigram 用詞庫先驗當回退、式 (8) 與 (9) 的替換、式 (10) 的下限，都沒有重新正規化，所以對 w 加總不一定是 1；pb、pb_{KN} 與 q 同樣是回退值。這些量只用來排路徑的先後，引擎沒有為了讓它們正規化而改。

3.5 語意校正：小型字元語言模型與融合規則

重排模型是六層 pre-LN Transformer 解碼器，寬 384、六頭、位置視窗 64 字、輸入與輸出共用嵌入、學習位置、GELU，字表 10,351 個 id；14,646,912 個參數以每列對稱 int8 加 float32 縮放存放，檔案 14,886,880 bytes。64 是模型的視窗；餵給它的前文、也就是鍵 k，最多兩個字。令 $y_0, \ldots, y_{n_c - 1}$（$n_c \le 8$）是一段解碼器前幾名的路徑、依解碼器順序，$ng_i = S(\pi_i)$ 是它們的路徑分數，k 是這一段之前最多兩個漢字（沒有就是空）。候選 i 的模型分數是

$$ll_i = \sum_{t=1}^{|y_i|} \log_{10} p_\theta\big(y_{i,t} \mid [\mathrm{BOS}], k, y_{i,<t}\big), \tag{3}$$

n_c 個候選右側補齊、一次前向傳遞。只有一個候選的段不重排。令 U_0 是可用的候選：候選 0 加上每一個沒被遮罩的 $i \geq 1$（候選會換掉解碼器第一名裡手動加的詞時被遮罩，v0.7.0 起重排保護詞表的五個常用詞也一樣 [51]；附錄 A.3 的模糊音規則也把含容錯詞的候選拿掉，第一名本身含容錯詞的段整段不重排）。用該強度的三元組 (α, τ, δ) 與附錄 A.8 的學習項 ℓ_i（$\ell_0 = 0$ 是定義使然），決策（「pick」）是

$$
\begin{aligned}
f_i &= ng_i + \alpha\,\tilde{ll}_i + (\ell_i - \ell_0), \qquad U = \begin{cases} \{0\} \cup \{i \in U_0 : \ell_i > \ell_0\} & ng_0 - ng_1 \geq \tau, \\ U_0 & \text{其他}, \end{cases} \\
j &= \arg\max_{i \in U} f_i, \qquad \text{選} \begin{cases} 0 & U = \{0\}, \\ j & f_j - f_0 \geq \delta, \\ 0 & \text{其他}, \end{cases}
\end{aligned}
\tag{4}
$$

其中書面設定 $\tilde{ll}_i = ll_i$，聊天設定是式 (6) 口語融合後的值。argmax 同分取解碼器名次較好的，所以 $\delta = 0$ 時候選要嚴格高於 f_0 才換掉第一名。$\ell \equiv 0$ 時就是：解碼器第一名對第二名的分差至少 τ 時不動；否則融合分數最高的候選只在贏至少 δ 時換掉它。三種強度是三組三元組，也是強度設定唯一改的東西。v0.7.0 的值，保守、標準、積極三檔依序是聊天 $(1.3, 2, 1)$、$(1.1, 2, 0)$、$(1.1, 2, 0)$，書面 $(0.8, 2, 1.5)$、$(0.8, 2, 0)$、$(0.8, 2, 0)$（v0.6.1 的在附錄 H）。門檻 τ 與 δ 是在指定的調參與判讀資料上選定的分差（第 4、5.1 節），不是校準過的機率。聊天三組放在核心（它們是帶口語項選的）；書面三組在模型檔頭。殼非同步套用選出的候選，那一段中途變了就丟掉結果；解碼器第一名含被學習抬起的詞的段不送重排（保護學到的詞）。

模型檔記著它調參時配的 bigram 與 Kneser–Ney 檔的 SHA-256，配到別的檔就拒絕載入、只用解碼器。訓練文字是維基（前 200,000 篇）、Common Voice zh-TW、Tatoeba、合成口語句、g0v `#rand0m`、論壇 A（台灣公開論壇的文章與推文），以及 OpenSubtitles v2018 繁體字幕 [29]（清理後 4,432,866 句、42,814,742 個漢字）。權重先從頭訓練（36,005 步、一個 epoch、AdamW、cosine、峰值 1e-3、200 步暖身），再接續訓練 3,000 步、峰值 1e-4，來源相同、重新轉換、不套 `TWPhrases` 這張轉換表（第 5.4 節）。

3.6 聊天設定的口語項

口語表是只在口語池上計數的普通 bigram 表（詞彙 $V_{\mathrm{CL}} = 69{,}457$，總次數 N_{CL}），讀的時候不帶詞類、不帶側檔，所以它的分數 P_{CL} 照式 (2)，用表自己的折扣，詞的回退值 $pb_{\mathrm{CL}}(w) = (c_{\mathrm{CL}}(w) + 1)/(N_{\mathrm{CL}} + V_{\mathrm{CL}} + 1)$（表外的詞是 0）；句尾項退回表自己的句尾比率。候選 i 的切分 $w_1 \cdots w_m$ 的分數是

$$
lc_i = \sum_{j=1}^{m} \log_{10} P_{\mathrm{CL}}(w_j \mid w_{j-1}) + \log_{10} P_{\mathrm{CL}}(\langle /s \rangle \mid w_m), \qquad col_i = lc_i - \kappa\, o_i, \qquad \kappa = 0.5, \tag{5}
$$

$w_0 = \langle s \rangle$，o_i 是切分裡來自疊加層或詞包、而且不在基底詞庫裡的詞數。出貨設定（「溫和版」）下，表裡沒有 w_1 的 $\langle s \rangle$ 二元組時，第一個詞的因子改用 $pb_{\mathrm{CL}}(w_1)$ 而不是 $P_{\mathrm{CL}}(w_1 \mid \langle s \rangle)$。這一項相對於解碼器第一名、經軟閘門進入融合，$\sigma(x) = 1/(1 + e^{-x})$：

$$
\tilde{ll}_i = ll_i + g_i\, \frac{\mu_{\mathrm{CL}}}{\alpha}\, (col_i - col_0), \qquad g_i = \sigma\Big(\frac{ll_i - ll_0}{T_g}\Big), \qquad \mu_{\mathrm{CL}} = 0.5,\ T_g = 1.0, \tag{6}
$$

所以在 $f_i = ng_i + \alpha\tilde{ll}_i$ 裡這一項的權重是 $\mu_{\mathrm{CL}}\, g_i (col_i - col_0)$，和 α 無關；每多一個詞包詞的扣分權重是 $\gamma = \kappa\mu_{\mathrm{CL}} = 0.25$。閘門讀 ll_0：字元模型自己就把某候選排在錨點之下時，它拿到的口語證據很少。任何一個 col_i 不是有限值時，這一段不加口語項，改用檔頭聊天三元組的原本規則。這一項在聊天設定三種強度都生效，而且只在載入的重排器就是量這些格的那一份時（以雜湊配對）。

3.7 改選學習

使用者改選一個詞時，引擎存一筆（前文鍵，讀音，詞）紀錄，權重隨時間衰減，之後解碼時把那個條目的先驗抬到 $lp'(w)$；被抬起的條目不扣讀音項，第一名含這種條目的段不送重排。句首的單字改在融合裡處理，也就是式 (4) 的 ℓ_i：教過的字會被加進候選池，並依後面的音節和教的時候相符的程度得到加分。公式在附錄 A.8；這條規則到第三版才通過單次確認（表 1 第 7–9 列）。

3.8 延遲與記憶體

本文有兩種延遲數字，都不是完整 App 的延遲。每鍵的數字來自跨版本基準測試：它把 302 句開發集的 8,336 個按鍵走同步的按鍵路徑重播，載的是引擎載入模型時會讀的那一版的檔（bigram 與詞類檔；那一版有的話，再加 Kneser–Ney 側檔、三元表與解碼器的讀音項），不載詞包、沒有前文、沒有語意校正 [51]，量的是精簡後的解碼器。重排的數字由評測 CLI 另外量。兩者都不含輸入法的殼、候選窗與 App。

在開發用的 Mac 上，v0.6.0 每鍵 p95 2.232 ms（聊天）、引擎載入 945 ms、RSS 480 MB、重排 p50 ／ p95 6.68 ／ 8.58 ms；發佈說明報 v0.5.0→v0.6.0 為 2.06→2.23 ms、892→945 ms、441→480 MB、8.3→8.6 ms。v0.6.0 那一次開跑時 load average 5.8，不是閒置窗的量測。同樣的重播在更高的負載下，較早的版本量到 5–8 ms，所以「約 2 ms」只描述負載輕的機器。解碼器的讀音項最多多 7.5 MiB、p95 差 −0.01 到 0.0 ms，這一項是在閒置窗量的（§rl-089）。

v0.7.0 這一次，基準測試在同一場裡讓 v0.6.1 與 v0.7.0 輪流重播（各三輪，取 p95 的中位數），用的是和上面幾次不同的一台 Mac，所以這組數字只能彼此比，不能和 v0.6.0 的比。開跑時 load average 1.9，在專案回報速度數字的上限 2.0 以內；結束時 4.2，速度是最後才量的，所以結束時的負載含它自己 [51]。從 v0.6.1 到 v0.7.0，每鍵 p95 聊天設定 1.91→1.93 ms、書面設定 1.92→1.96 ms，引擎載入 0.87→1.05 秒，RSS 480→489 MB，重排 p50 ／ p95 5.56 ／ 7.15→5.53 ／ 7.11 ms [47, 50]。

4 評測方法

下面常用幾個詞。「實驗」是一個有自己書面契約或登記的工作單位；「格」是參數格點上的一點，對重排模型是一組三元組 (α, τ, δ)；「單次確認」是在那個實驗沒看過的資料上只跑一次、之後那份資料退役；「改對」「改壞」是一個改動對基準由錯變對、由對變錯的列，「淨值」是改對減改壞，「比例」是改壞 ÷ 改對，「流動」是兩個本該等價的系統互換的列數；$U_2 =$ 改對 $- 2 \times$ 改壞 是挑重排模型訓練用的分數。「獨立的 agent」是沒有參與被審查那項實作的 LLM session（見引言）。

4.1 指標

主要指標是句層級的第一名正確率（top-1），用寬鬆對照：把「她妳它牠嘗周臺裏」對應成「他你他他嚐週台裡」，並把 2,072 個辭典明列「也作」、讀音相同的異體詞視為相同。這是專案一開始看過開發集錯句後採用的事後規則，但標準定死在辭典、不從測驗句挑（§me-001）。次要指標：oracle@64（正解在前 64 名候選裡）、字元錯誤率（第一名對正解的 Levenshtein 距離除以正解字數）、正解在 64 名候選分數 softmax 下的對數損失，以及延遲。另報一個也接受辭典記載替代寫法的「合法寫法」分數，從不用來判讀。表格裡數字後的「a/b」是改對／改壞。

4.2 集合與角色

表 8（附錄 B）列出集合。三個性質決定它們的組織。(i) 句子是在標點處切的子句，前文是同一則訊息的前一個子句。(ii) 讀音由工具從正解產生（單字取主要讀音），只有錯字回報集的讀音是使用者實際打的；這個差別有影響（第 8 節）。(iii) 每個集合同一時間只有一個角色；下面是 2026-10-18 分層之後的角色，之前有兩個換過：discordtune 原本是主要的調參集、之後才成為開發層的檢查

集，dev302 從兩種設定的回歸關卡改成只擋書面。回歸層（dev302、typing76、probe、功能字、wikitypo、錯字回報）要逐列相同或最多改壞 2 列且淨值非負；調參層（cvtune-clean、wikitune-clean、chatclause ／ `ctune`）是選值的地方；開發層（discordtune ／ `cdev`、lytune、newstune）是檢查非劣性的地方；判讀層（chatjudge-k、wikijudge-k、`cjudge`-k）每份只用在一個決定、用過就退役；保留集（150 個公開口語子句與 150 個私人新聞子句）只由獨立的 agent 在推 tag 前量，記使用次數，用滿八次換下一份（§me-002、§me-014；[48]）。

保留集是什麼。 保留集是發版監測用的集合，不是從沒看過的測驗。口語半邊（150 個公開子句，聊天設定）從 2026-10-16 現役到 2026-11-11，它的統計數字回報過八次：前四次它還是 300 列合併份的一部分，之後是 v0.6.0 發版（第五次）、v0.6.1 發版（第六次），v0.7.0 量了兩次（第七、第八次：加進重排保護詞表之後重量一次，摘要行相同）。實作的 agent 從不讀它的列，只拿到摘要行；一個半邊用滿八次、或有任何決定因它的數字而做，就換下一份；2026-11-11 口語半邊用滿八次，換成還沒用過的下一份。新聞半邊（150 個私人子句，書面設定）在 v0.6.0 發版時新啟用，那是它第一次使用，v0.6.1 是第二次，v0.7.0 的兩次是第三、第四次。原本 227 列的保留集到 v0.4.6 為止每次發版都量，幾個部件的實驗也量過它（附錄 C）；公開紀錄沒有它的使用次數，它現在是可讀的挑戰集 [48]。所以本文把保留集的數字當監測證據引用，確認性的證據只取表 1 的單次確認。

4.3 配對統計與判讀的依據

比較是對基準逐列配對，報改對、改壞與淨值。用兩種檢定：不一致列上的精確雙尾 McNemar 檢定 [12]，抽樣單位是子句；以及 Δtop-1 的配對 bootstrap 區間 [25]，報告用 95%、非劣性用 90%。

各時期的主判讀。 決定一個實驗過不過的統計量換過一次，下面逐期寫明。

- *2026-10-18* 以前。實驗有書面契約，通常是某個指定集合上淨值為正且精確 McNemar $p < 0.05$，再加幾條防線（改壞率上限、其他集合不顯著變差）。主判讀沒有在量測之前 commit，bootstrap 以子句為單位重抽，有些條件在看過結果之後改過（附錄 E）。這段時期的 p 我們一律當描述統計。
- *2026-10-18* 起。確認在讀任何數字之前先 commit 一個主判讀。主張「有進步」時的規則是：淨值 > 0、整份判讀集上精確雙尾 McNemar $p < 0.05$、比例 ≤ 0.65。主張「沒有變差」時用非劣性界值：Δtop-1 的 90% 配對 bootstrap 區間下界不低於 -1.0 個百分點。從這一天起 bootstrap 以叢集為單位重抽：同一則訊息、同一段發言或同一個條目切出的子句一起抽。紀錄有每個集合的子句數，沒有叢集數。
- 改壞率上限用改壞數的單尾 95% 精確 Poisson 上界除以 n，不用點估計，因為點估計兩次選到在調參資料上壓線、到新資料上超過的值（0.50%→1.72%；0.43%→0.54%；§me-014）。

表 1 的「規則」欄寫明每一次確認用的是哪一種。

描述性的 p 與確認的門檻。 本文裡只有「單次確認事先 commit 的主判讀」的 p 是確認性的。其他的 p（相鄰版本的比較、部件在調參層與開發層集合上的比較）都是描述性的：那些集合已經看過，同一批集合上做了很多次比較、沒有多重比較校正；Holm 只用在餵進同一個登記決定的那一組檢定（§me-004）。就算是確認性的 McNemar p 也偏樂觀，因為檢定把子句當成獨立，而同一則訊息或同一個條目切出的子句彼此相關。我們沒有用叢集檢定重新分析過去的確認，因為那是事後分析；這些確認照當時判讀的方式報告。

「不顯著變差」不等於「達到非劣性界值」。 下面的出貨標準用到兩種不同的說法，本文把它們分開。不顯著變差（精確 McNemar $p \geq 0.05$，或淨值 ≥ 0）是沒有偵測到損失；它不替損失設上限，不一致列少的時候很容易過。達到非劣性界值替損失設了上限：Δtop-1 的 90% 下界要高過事先訂好的界值才算。前一個說法只用在前者，後一個只用在後者。

解析度。 開確認之前，用基準的不一致列數算最小可偵測效果：不一致 61 列時，$\alpha = 0.05$ 的 80% 檢定力淨值是 +23 列，Holm 第一關 $\alpha = 0.01$ 是 +27 列。小集合上，配對精確符號檢定要改對對改壞約 6:0 或 8:1 才有 $p < 0.05$，所以 227 列或 150 列的集合上差一句是雜訊。

4.4 探索與確認

2026-10-18 起，規則把實驗分成兩種。探索是一頁登記（假設、指標、集合、預算、停止條件），只碰調參層與開發層的集合，不出貨，結論只寫方向。確認在讀任何數字之前先 commit 一個主判讀、它的規則、集合、預算、最小可偵測效果與診斷，在沒看過的判讀集上只跑一次。沒過就寫「照登記不過」；不另開實驗救同一次確認，變體回探索，再確認要新的判讀集。看過結果後改門檻、篩選、集合或格點，那次確認作廢。「契約不要訂得過嚴」只適用產品與 UX 規則，不適用統計判讀規則（§me-014）。規則在看過數字之後才改的地方，本文都寫明；受影響的改動在還有新集合可用時重新確認過。唯一沒有新集合可用的那一次、沒有單次確認就出貨的改動，以及其他偏離這些規則的地方，列在附錄 E。

4.5 出貨標準

新模型、或任何改到選字的改動，對現行出貨要同時滿足：

1. 在聊天開發集（discordtune，之後是 CE3 `cdev`）不顯著變差（精確 McNemar $p \geq 0.05$ 或淨值 ≥ 0），而且改壞 ÷ 改對 ≤ 0.6；
2. 回歸層（兩種設定的 typing76、書面設定的 dev302）逐列相同或改壞 ≤ 2 且淨值 ≥ 0；聊天設定的 dev302 照報、每個改壞列標類別，不擋；
3. 聊天設定在 cvtune-clean（後來換成 cvtune-clean-tw0）的 Δtop-1 90% 配對 bootstrap 下界 ≥ -1.0 個百分點；
4. 錯字回報集兩種設定合計淨值 > 0 且改壞 ÷ 改對 ≤ 0.6（只加詞的詞包改壞 0 算過，含 0/0）。

第 1 條是第 4.3 節說的「不顯著變差」，第 3 條是非劣性界值。照規則，四條都過的探索，做成產品改動之前先在沒看過的判讀集上確認一次；沒有做這一步就出貨的改動列在附錄 E。第 3 條有一個已知的弱點：cvtune-clean 是調參層的集合，出貨的 (α, τ) 就是在它上面選的，所以基準在它上面有主場優勢、這一條偏寬；我們把它當成「非劣性檢查用開發層」這條規則的例外，在它上面選過值的改動不再拿它來把關（§me-017）。這套標準本身改過幾次，其中兩次是看過數字之後才放寬；只在放寬後的規則下過關的改動，之後在一份新的聊天判讀集上確認過（表 1 第 3 列）。帶日期的沿革在附錄 C。出貨的改動有一條沒過、以記錄在案的例外放行的，附錄 C 寫明並引用紀錄。工程上的控制（同一次比較用同一種 CPU、確認的量測在第二台機器重跑、每個輸入檔核對雜湊）在附錄 E。

4.6 出處與可重現性

除了一個例外，本文每一個數字都取自專案公開的文件，沒有為了這篇論文重算：研究摘錄 [49]（開發紀錄裡本文引用到的段落，逐段公開）、跨版本基準測試 [47]、評測集說明 [48]、規格 [51] 與

發佈說明 [50]。寫成 §rl-012 這種形式的引用，指摘錄裡同名的錨點。摘錄以 commit `5a33e0ad` 為準：它在 v0.7.0 那個 tag 之後，補了計畫的四段（§plan-019～§plan-022），原有的段落沒有改動；其餘四份以 v0.7.0 那個 tag 的 commit 為準。表 2、9、3 的 v0.7.0 列、圖 2 的 v0.7.0 那一點與 v0.7.0 的延遲，取自基準測試那一版的結果檔、保留集的使用紀錄與發佈說明；v0.7.0 的百分比是公開的列數除以集合大小。例外是表 4：它的量測是為這篇論文在打了 tag 的 commit 上跑的（第 5.3 節）；結果摘要與離線重算的程式 `gate_offline.py`（分差閘門那一列用 `--off gate`，軟閘門那一列用 `--off soft`）在 repo 的 `eval/ablation/v0.7.0/`（commit `e88f4a36`）[46]。程式、詞庫與模型檔都在 repo [35]；公開的調參集與回歸集可以用評測 CLI 重跑，模型檔依雜湊抓取。私人評測集（作者自己的聊天訊息，以及一份來源未授權再散布的新聞集）只寫大小與文件已公開的統計數字，沒有任何一句出現在這裡。來源標成推論而非量測的句子，保留它的標記。

5 結果

這一節照三個研究問題排。第 5.1 節是確認性的證據；第 5.2 節是各版在開發集與保留集上的趨勢；第 5.3 節是 v0.7.0 系統的逐項拿掉；第 5.4 節是正確率靠什麼動、量測有多可靠；第 5.5 節是兩個外部比較。每個部件的沿革（含專案代號與日期）在附錄 C。

5.1 單次確認

表 1 列出專案跑過的每一次單次確認：一個主判讀一列，同一次確認的排在一起。這張表只整理既有的紀錄。13 次確認裡 8 次成立、5 次不成立；以主判讀計，17 個裡 12 個過。不成立的 5 次裡，有 4 次跑之前在調參或開發資料上是過的；另一次（第 5 列）登記的臂不是先前量過的那一臂。

表的說明。第 1 列主判讀過了、改壞率的防線沒過，所以那一格沒有匯出；改壞率上限就是因為這一次才改成用上界。第 2 列的 0.2% 那一格，是把上限在看過的資料上放寬到 0.6%（有寫明）之後才進確認。第 5 列用的是調參層的集合，只是這條線沒看過它；登記的臂也不是先前量過的那一臂（附錄 E）。第 7–9 列的集合是為這個目的寫的教學句、罕用句與常用句。第 10 列的那一臂確認成立但沒有出貨；出貨的是第 11 列。第 12、13 列的聊天設定格是在維基文字上確認的，因為聊天判讀集已經被第 6 列用掉；它們能不能轉移到聊天，只有出貨標準第 1 條支持。紀錄有判讀集上絕對 top-1 的幾列是：第 4 列 2,231→2,250；第 10 列 2,262→2,260；第 11 列 2,255→2,254；第 12 列 2,119→2,164（書面）與 2,045→2,071（聊天）；第 13 列 2,100→2,149（聊天）與 2,208→2,222（書面）；其餘各列紀錄只有配對的列數（§rl-039–§rl-042、§rl-053、§rl-076、§rl-077、§rl-087、§rl-088、§rl-090–§rl-094、§rl-108、§rl-110、§rl-113、§rl-114）。

沒出現在表裡的判讀集：wikijudge3 只在紀錄裡和前兩份一起被列為已退役（§ip-005），用在哪個決定，紀錄沒有。第一份聊天判讀集在紀錄裡是舊的聊天集之一，也是一項診斷的資料來源（§me-021、§ip-002）；紀錄沒有在它上面跑過單次確認。在這套做法開始之前，v0.4.1 的重排融合格 (α, τ, δ) 在調參月份上選、在保留下來的一個判讀月份（6,559 列）上計分（附錄 C）；那不是 commit 過的登記，這裡不計入。

5.2 各版在開發集與保留集上的結果

表 2 取自 repo 的跨版本基準測試 [47]：每一版用自己出貨的模型與旗標在凍結的集合上跑。表裡每個集合在選值時都被看過，每一版又綑綁了幾個改動，所以這張表描述的是版本，不是檢定。基準測試裡描述性的 p 這裡不列；每一版的改對與改壞在表 9（附錄 C）。圖 2 畫的是兩個 discordtune 欄，表 3 是保留集。

這幾張表裡有三件事和研究問題有關。第一，最大的一步是 v0.6.0，也就是用論壇與聊天頻道文字重建 bigram 與重排模型的那一版：聊天開發集改對 336、改壞 49，凍結的錯字回報 80 列從 33 列對到 70 列對（第 5.4 節）。第二，這一代在真正的書面文字上沒有更好：維基調參集上書面

表 1: 單次確認，照執行的先後排。規則 I（有進步）：淨值 > 0、精確雙尾 McNemar $p < 0.05$、比例 ≤ 0.65。規則 N（非劣性）：Δtop-1 的 90% 配對 bootstrap 區間下界 ≥ -1.0 個百分點。「改對／改壞」對的是附錄 C 寫明的那一臂。†：2026-10-18 以前，依書面契約跑，沒有 commit 過的登記。「成立」說的是整次確認。

#	確認的改動	沒看過的集合（列數）、設定	規則	改對／改壞	p 或界值	成立
1†	書面融合格，第一次重選	wikijudge（列數紀錄沒有），書面	主判讀加改壞率上限 0.5%	76/21	$p < 0.0001$；改壞率 1.72%	否
2†	書面融合格，第二次重選	wikijudge2 (2,818)，書面	主判讀加改壞率上限	39/11	$p = 0.0001$；0.35%	是
	保守格	同上	同上	20/5	$p = 0.004$；0.14%	
3	口語表過軟閘門	chatjudge-2 (5,783)，聊天	I	108/39	$p = 1.1 \times 10^{-8}$；比例 0.36	是
4	放寬的書面融合格	wikijudge4 (2,820)，書面	I	67/48	$p = 0.0928$；比例 0.72	否
5	跳一個詞的詞類橋接	過濾後的 chatclause (5,631)，聊天	淨值 > 0、改壞 $\leq$ 改對的一半	0/2	—	否
6	解碼器裡的讀音項	wikijudge5 (2,816)，書面	I	15/0	$p = 6.1 \times 10^{-5}$	是
		cjudge-1（5,639），聊天	N	3/4	下界 −0.089 個百分點	
7	句首單字學習，第一版	專案寫的句組	常用列由對變錯 ≤ 1	罕用列對 24 比 7	3 列變錯	否
8	第二版	第二組	同上	21 比 9	2 列變錯	否
9	第三版	第三組	同上	13 比 7	1 列變錯	是
10	重排模型接續訓練，第一臂	wikijudge6 (2,844)，書面	N	9/11	下界 −0.32 個百分點	是
11	重排模型接續訓練，出貨的那一臂	wikijudge7 (2,857)，書面	N	5/6	下界 −0.25 個百分點	是
12	新融合格，標準檔	wikijudge8 (2,844)，書面	I	90/45	$p = 0.0001$；比例 0.50	是
		同一份，聊天設定	I	43/17	$p = 0.0011$；比例 0.40	
13	新融合格，保守檔	wikijudge9 (2,827)，聊天設定	N	68/19	下界 +1.20 個百分點	是
		同一份，書面	N	14/0	下界 +0.28 個百分點	

設定 2266→2243（+78/−101）、聊天設定 2184→2160（+80/−104），描述性的檢定都不顯著；資訊詞包檢查集也退步：重建的第一階段（新的 bigram 配上一代的重排模型）從 46 掉到 36（聊天）、39 掉到 27（書面），發佈說明引的是這一組數字；出貨組合的定案判讀是改對 3、改壞 14（聊天）與改對 4、改壞 10（書面）（§rl-059、§rl-068；[50]）。更早的 v0.4.4 在立法院發言的集合 lytune 上退步（聊天 4055→4010、書面 4021→3979）；原因是兩個解碼器項的交互作用，兩版之後大致修好（第 5.4 節）。第三，開發集與保留集的水準差很多：v0.6.0 在作者聊天的 4,958 個子句裡對 4,645 個，在公開口語半邊的 150 個子句裡對 124 個。這兩個集合的來源與語域不同，被選值看過的程度也不同，沒有實驗把這兩種影響分開，所以我們不把這個差距讀成量到的過度配適，也不排除有過度配適。紀錄裡有一筆數字不支持「純粹是選值造成」的說法：較早的一版流程在 discordtune 是 86.71%，在沒有用來選值的判讀月份是 88.61%（第 5.5 節）。

這幾張表的最後一列是 v0.7.0，也就是本文描述的那一版。對 v0.6.1，聊天開發集在聊天設定改對 31、改壞 12，書面設定改對 100、改壞 42。書面設定在維基調參集（2243→2303，+97/−37；

表 2: 跨版本基準測試：對的列數（top-1，寬鬆對照），沒有另外註明的都是聊天設定。「錯字回報」是凍結的 80 列；最後一欄是同一批 discordtune 子句在書面設定下的結果。版本從 v0.4.0（2026-10-10）到 v0.7.0（2026-11-11）。所有集合在選值時都被看過。

版本	dev302 (302)	typing76 (76)	錯字回報 (80)	cvtune (3,791)	discordtune (4,958)	同左，書面
v0.4.0	241	66	17	3360	4209	4143
v0.4.1	259	69	25	3441	4299	4172
v0.4.2	259	69	25	3441	4299	4172
v0.4.3	259	69	25	3445	4314	4191
v0.4.4	259	70	24	3446	4303	4177
v0.4.5	259	70	24	3446	4303	4177
v0.4.6	262	72	28	3448	4304	4178
v0.4.7	262	72	28	3448	4304	4218
v0.5.0	269	72	33	3485	4358	4267
v0.6.0	280	74	70	3530	4645	4588
v0.6.1	280	74	70	3530	4645	4588
v0.7.0	282	74	70	3536	4664	4646

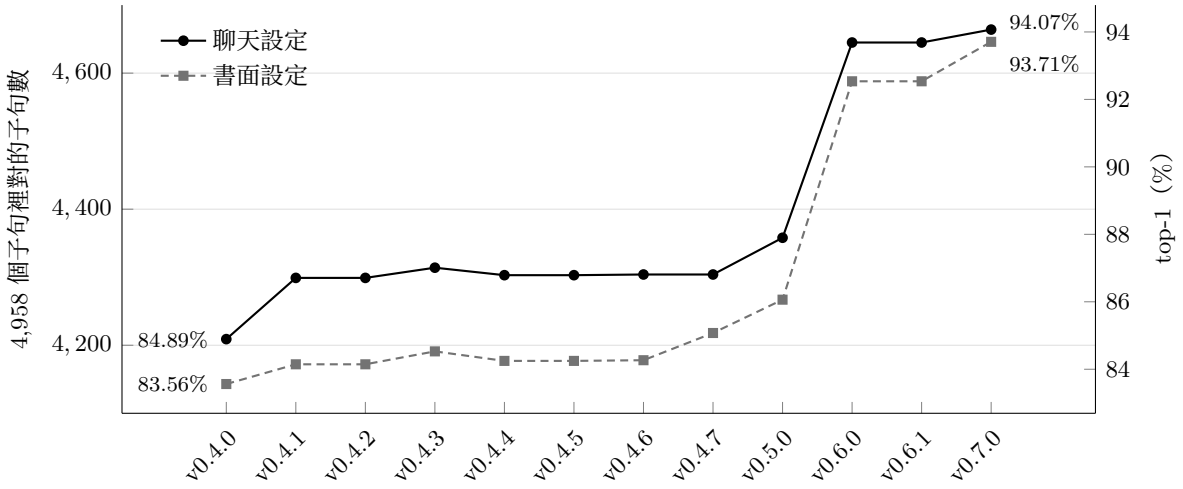


圖 2: 各版在 discordtune（作者聊天的 4,958 個子句）上的第一名正確率：聊天設定，以及同一批子句的書面設定。每一點都是表 2 兩個 discordtune 欄裡的一格；右軸把同樣的列數換算成佔 4,958 的百分比，標出的四個百分比是摘要與結論引用的頭尾兩點。discordtune 是開發層的集合，選值時看過（表 8），所以這兩條線是描述性的；每一版綑綁了幾個改動。

聊天設定 2160→2186，+46/−20）、dev302（271→280，+10/−1）與凍結的錯字回報 80 列（61→69，+9/−1；聊天設定維持 70）也上升。有兩個公開集合往反方向走：立法院發言的集合 lytune 在聊天設定（4430→4401，+33/−62；書面 4384→4403，+114/−95），以及維基錯字集在書面設定（155 列裡 137→131，+1/−7）。保留集口語半邊從 124 到 127、新聞半邊從 128 到 126，兩個差距都不顯著（$p = 0.25$、0.63；精確檢定約要改對對改壞 6:0 或 8:1 才顯著，第 4.3 節）。其他集合的描述性 p 值在附錄 C [47]。

表 3: 保留集，每次打 tag 前由獨立的 agent 量（發版監測的證據，見第 4.2 節）。到 v0.4.6 為止是原本的 227 列；v0.4.6 起是換新的 300 列（新聞加口語）；v0.6.0 起是各 150 列的兩個半邊（公開口語，聊天設定；私人新聞，書面設定）。第二個 v0.5.0 列是同一版在兩個半邊上、較晚的 commit 上量的，當 v0.6.0 的基準。「第幾次」是公開紀錄裡口語半邊與新聞半邊的使用次數；v0.7.0 量了兩次（加進重排保護詞表之前與之後，結果相同），之後口語半邊換份（第 4.2 節）。換份前後的數字不能相比。改對／改壞與精確 McNemar p 對的是上一個打了 tag 的版本；沒有變化的 v0.4.2、v0.4.5 不列。

版本	集合	聊天	書面	第幾次（口語／新聞）
v0.4.0	227	180 (79.30%)	184 (81.06%)	紀錄沒有
v0.4.1	227	191 (+11/−0, $p = 0.000977$)	187 (+3/−0, 0.25)	紀錄沒有
v0.4.3	227	191 (0/0)	189 (+2/−0, 0.5)	紀錄沒有
v0.4.4	227	191 (+3/−3)	186 (+2/−5, 0.453)	紀錄沒有
v0.4.6	227 / 300	191 (+3/−3) / 234	187 (+6/−5) / 230	紀錄沒有
v0.4.7	300	234 (0/0)	233 (+7/−4, 0.549)	1
v0.5.0	300	237 (+7/−4, 0.549)	241 (+8/−0, 0.0078)	4
v0.5.0	150+150	115 (76.67%)	125 (83.33%)	和 v0.6.0 同一次
v0.6.0	150+150	124 (+10/−1, 0.0117)	128 (+3/−0, 0.25)	5 / 1
v0.6.1	150+150	124 (0/0)	128 (0/0)	6 / 2
v0.7.0	150+150	127 (+3/−0, 0.25)	126 (+1/−3, 0.625)	7, 8 / 3, 4

5.3 v0.7.0 系統的逐項拿掉

第 5.4 節每個部件的結果，都是在它引入當時、對當時那一代模型量的，所以不能相加，也回答不了「現在的系統裡每一塊各貢獻多少」。表 4 回答這個問題：在 v0.7.0 那個 tag 的 commit 上、標準檔，每一列用評測 CLI 現有的開關拿掉一塊。讀這張表要先知道三件事。「拿掉口語項」時，融合改用模型檔自己的聊天三元組 $(0.3, 2, 0.25)$、不是聊天那一格 $(1.1, 2, 0)$（聊天的三元表照開），所以這一列是口語項加它那一格的合併效果。詞類與 Kneser–Ney 兩列是對只有解碼器的系統拿掉，因為重排模型載入時 CLI 不讓它們被拿掉：重排模型的格是在它們開著時選的。兩個閘門列沒有開關，是離線重算的：用完整系統那一跑的逐候選分數，照式 (4) 與式 (6) 的出貨規則（先核對過這條規則在每一列都重現那一跑的選擇），分差閘門那一列把 τ 設成 ∞、δ 設成 0，軟閘門那一列把 g_i 固定為 1，其他照舊。這些量測是描述性的，也是唯一為這篇論文算的數字。

從這張表可以讀出四件事；下面的 p 都是描述性的精確 McNemar 值。第一，在逐項拿掉的各塊裡，重排模型（連它的口語項）在三個集合上都是最大的一塊（n-gram 模型本身不在逐項拿掉之列；只用解碼器的系統和最後一列的 unigram 基準之間的差距大上好幾倍，而那個基準另外也不帶詞包）：拿掉之後淨少 136、124、117 列（都是 $p < 0.0001$）；只拿掉口語項（連它那一格），兩個聊天集合淨少 74 與 51 列（$p < 0.0001$）。第二，解碼器的各塊因語域而異。拿掉三元表，書面設定淨少 60 列（$p < 0.0001$），聊天看不出差別（-3，$p = 0.51$；-5，$p = 0.23$）。在只有解碼器的系統裡，拿掉 Kneser–Ney 接續值淨少 29、43、17 列（$p = 0.0002$、< 0.0001、0.036），拿掉詞類淨少 3、9、3 列（$p = 0.55$、0.049、0.63）。第三，拿掉讀音項沒有改對任何一列，改壞 8、22、17 列（$p = 0.0078$、< 0.0001、< 0.0001）；這些集合的讀音是工具產生的，讀音項扣次要讀音的代價在上面顯現不出來（第 8 節）。第四，在標準檔，分差閘門在三個集合上都幾乎從來不是真正擋住的那個條件：它在公開聊天集 2,992 列裡的 802 列、書面 2,182 列裡的 509 列、私人聊天集 4,919 列裡的 2,012 列保住解碼器的第一名，而沒有它時，融合在第一份一列都不會改、在第二份改兩列、在第三份改一列，三列都改壞（$p = 0.5$ 與 $p = 1$）。這一列只量到門檻 τ，因為標準檔的格本來就是 $\delta = 0$；保守檔的 δ 沒有拿掉來量。把口語項的軟閘門固定為 1，公開的聊天集從 2,771 列對變成 2,775 列對（改對 11、改壞 7，$p = 0.48$），私人的聊天集從 4,633 變成 4,628（改對 5、改壞 10，$p = 0.30$）；聊天那一格是在軟閘門開著時選的，所以這一列只能當描述，它沒有說明不帶

表 4: v0.7.0 系統的逐項拿掉（commit `56979b2e`，v0.7.0 的 tag）：對的列數（top-1，寬鬆對照），括號裡是對該區塊第一列那個系統的改對／改壞。標準檔、出貨旗標（前文與三個預設開的詞包）、在一台機器上跑一次。cvtune-clean-tw0 與 wikitune-clean-tw0 是公開集合；discordtune 是過濾後的私人集合，只報彙總數字，分數檔不公開。書面設定沒有口語項，所以是「不適用」；unigram 基準走另一條程式路徑，不帶詞包、不寫逐列結果，所以沒有改對／改壞。三個集合在選值時都被看過，這張表是描述性的。

系統	cvtune-clean-tw0 （聊天，2,992）	wikitune-clean-tw0 （書面，2,182）	discordtune （聊天，4,919）
完整系統	2,771	1,677	4,633
拿掉重排模型與它的口語項	2,635 (21/157)	1,553 (43/167)	4,516 (39/156)
拿掉口語項	2,697 (19/93)	不適用	4,582 (34/85)
拿掉分差閘門（$\tau=\infty$、$\delta=0$）	2,771 (0/0)	1,675 (0/2)	4,632 (0/1)
拿掉軟閘門（$g_i=1$）	2,775 (11/7)	不適用	4,628 (5/10)
拿掉三元表	2,768 (3/6)	1,617 (14/74)	4,628 (3/8)
拿掉讀音項	2,763 (0/8)	1,655 (0/22)	4,616 (0/17)
只用解碼器（第二列）	2,635	1,553	4,516
拿掉詞類回退	2,632 (4/7)	1,544 (4/13)	4,513 (7/10)
拿掉 Kneser–Ney 接續值	2,606 (15/44)	1,510 (17/60)	4,499 (21/38)
unigram 基準（只用詞庫，不帶詞包）	2,186	953	3,884

軟閘門調出來的系統會怎樣。這兩列都沒有動到融合權重 α、替換分差 δ，以及口語表從哪裡進入決策，而第 5.4 節問題二底下的結果變動的正是這幾件事。

5.4　正確率靠什麼動，量測有多可靠

表 5 整理各部件的實驗。每一列拿一個改動和當時出貨的版本比，用的是決定它的那個集合；這些列橫跨五代模型，所以效果是歷史上的比較，不能相加。最後一欄指向表 1 裡對應的確認（有的話）。

問題一：語域對的文字對上更大的神經模型。　表 2 裡最大的那一步，是把一個公開聊天頻道與一個台灣公開論壇的文字加進 bigram 的計數與重排模型訓練文字的那一代模型。重建的 bigram 單獨就在聊天開發集改對 201 列、改壞 33 列，錯字回報集從 58 列對到 95 列對（聊天）、51 到 99（書面）；用同樣來源重訓的重排模型，在一份調參集上以 U_2 比三個種子，三個都贏，而照舊配方重跑的三個種子都輸；它在新解碼器之上再改對 88 列、改壞 26 列。這些比較用的是開發層、回歸層與調參層的集合，都是看過的；這一代模型沒有單次確認（附錄 E）。用交叉熵差只挑新文字裡分數較好的一半，比全收差。代價付在書面文字上：同一個重排模型在維基調參集上輸了（1,634→1,616，25/43，$p=0.039$），我們把這讀成口語訓練文字變多的代價（沒有另外驗證）；書面回歸集有六列改壞，以記錄在案的例外出貨。相對地，把神經部件做大的嘗試沒有一個通過（第 6 節）。小型重排模型本身值得有：對只用解碼器，它在聊天開發集多對 67 列，遠少於在公開集合上的進步（dev302 聊天 241→277）；我們把這歸因於解碼器在那個集合上已經 84.9% 對（沒有另外驗證）。解碼器結構帶來的是較小的幾步：詞類與 Kneser–Ney 接續值在調參集上各值幾十列，兩者的交互作用造成唯一一次顯著的發版退步；把詞類權重調低之後，書面的損失全部補回、聊天的補回約六成。

問題二：決策規則。　有三個結果說明融合怎麼設定，和融合後面的模型一樣重要。(i) 在公開書面集合上選的值兩次沒能轉移到聊天；在作者自己的語域上重選那組三元組，判讀月份上改對

表 5: 各部件的實驗，各自對當時出貨的版本（歷史比較，不能相加）。「集合」寫集合與它的角色。沒有確認的列，p 是描述性的精確 McNemar 值。「確認」是表 1 的列號。

部件	集合（角色）	效果	p 或界值	確認
詞類回退	cvtune（調參）	淨值聊天 +21、書面 +26	0.002	—
Kneser–Ney 接續值	四格調參集，13,236 列	淨值 +33；wikitune 書面 38/20	0.025（那一格）	—
字元語言模型重排，1,440 萬參數	discordtune（聊天）	+67 列，95% 區間 $[29, 105]$	0.0008	—
在作者的語域上重選融合格	判讀月份，6,559 列	125/16（先前的值 82/7）	—	—
三元表，書面設定	wikitune（調參）	108/61	0.0004	—
詞類權重降到 0.2	lytune（開發）	聊天 100/73；書面 130/65	0.048；< 0.0001	—
重選書面融合格	wikijudge2（判讀）	39/11	0.0001	2
口語表過軟閘門	discordtune；chatjudge-2	74/43；108/39	0.0053；1.1×10^{-8}	3
用論壇與聊天頻道文字重建 bigram	discordtune（開發）	4,366→4,534，201/33	—	—
用同樣文字重訓重排模型	discordtune（開發）	4,558→4,620，88/26	—	—
解碼器裡的讀音項	wikijudge5；cjudge-1	15/0；3/4	6.1×10^{-5}；下界 -0.089 個百分點	6
重排模型接續訓練與新融合格	discordtune（開發），標準檔	29/11	0.0064	11–13
句首單字學習	第三組測試句	罕用列對 13 比 7	—	9

125、改壞 16，先前的值是 82 對 7。(ii) 口語表當成獨立的解碼器項時，在解碼器那一層改對 168 列、改壞 89 列，但整條流程重新計分之後在聊天開發集改壞 92 列；把它移進融合、以解碼器第一名為錨、過軟閘門之後，開發集是 74/43，沒看過的聊天判讀集是 108/39。只把融合調強、不加表的對照臂，判讀集 61/38、開發集 44/52，所以我們把進步歸給表而不是調強的融合（從對照臂推論，沒有另外驗證）。(iii) 最後一個重排模型：全面掃描發現，在舊的書面格之下，出貨的權重與新權重幾乎一樣（4,182 列裡 3,356 對 3,349），換成較強的格兩者進步也差不多（3,420 對 3,424），書面的進步幾乎全來自融合格。新的格之後在兩份沒看過的集合上確認（表 1 第 12、13 列）。

問題三：重訓的流動與選值的樂觀。 為了拿掉一張有問題的轉換表而從頭重訓重排模型時，三個種子都輸給出貨的權重。同種子的成對比較顯示，輸的幅度不超過重訓的流動：任兩個重排模型在聊天調參集互換約 90–100 列、在聊天開發集互換 65–80 列，淨值在 ± 12 以內（$p > 0.2$）。從出貨權重接續訓練把互換壓到 21–33 列；再試其他配方與種子，也沒有壓到種子雜訊底線以下（接續訓練的模型對它自己換種子的版本互換 39 與 33 列）。另一項檢查發現出貨的融合格是替某一個種子選的：同一份配方換種子，在選值集上對它 U_2 是 -50 與 -63。所以幾十列的模型改動，除非多跑幾個種子、每一臂各自重選格，否則和換一個種子分不出來。同一批紀錄也看得到選值的樂觀：選出來的格，改壞 ÷ 改對在選值集是 0.33、到開發集 0.58；下一個是 0.45 與 0.60；放寬的書面格在看過的資料上是 0.42、到判讀集 0.72；調參資料上 0.50% 的改壞率到新資料變成 1.72%。表 1 是總帳：13 次確認有 5 次不成立，其中 4 次在看過的資料上是過的。

轉換表是一類錯字的根因。 有兩列使用者回報的錯字，原因是重排模型的訓練文字被一個中文轉換工具的詞彙表轉過（OpenCC [9] 的 `TWPhrases`）；這張表會改寫 127 個詞庫裡的詞，其中 71 個是常用詞。在 94 句構造的句子上，出貨的重排模型把解碼器原本對的 49 列改壞 7 列（聊天）、46 列改壞 7 列（書面），用沒轉過的文字接續訓練的模型是 1 列與 0 列。同一張表也轉過調參集

的正解，這些集合之後不套它重建。這是資料的缺陷，不是模型的結果，但它改到的使用者看得到的列，比好幾個模型實驗還多。

5.5 外部比較

0c 模型在善解的候選池上當重排模型。 這個比較只回答一個窄的問題：把 MIT 授權的模型 `0c-sequence-d512` 放到善解重排模型的位置，在同一個候選池（解碼器最好的 8 條路徑）上做融合，選得會不會比較好？它不是兩個輸入法之間的比較。兩邊看到的輸入不同：0c 看前文最後 64 字加讀音；善解的重排模型看最多兩個字的前文，看不到讀音。第一次在 v0.4.1 的解碼器上，兩個模型的融合值用同一個方法選。在作者的聊天上，0c 的融合比當時出貨的重排模型好（discordtune 89.27% 對 86.71%，+189/−62；判讀月份 90.78% 對 88.61%，+200/−58）；在公開集合上（聊天設定），結果看集合而定：三個大的集合 0c 的融合較差（wikitune 73.65% 對 76.76%，+46/−134；lytune +191/−247；cvtune +54/−62，不顯著），dev302 相同（各對 259 列），幾個小的集合較好（錯字回報前 84 列 36 對 25，+16/−5，$p = 0.027$；typing76 72 對 69；功能字 161 對 157；維基錯字集 136 對 134）。檔案大小是 fp32 的 130.7 MB 對 int8 的約 14.6 MB（§ip-002；[47]）。第二次在 v0.6.0 的解碼器上，0c 的融合（格在聊天調參集上以 0.4% 上界選）對的是出貨的整條流程；出貨的流程除了重排模型，融合裡還有口語表與一個讀音項。在過濾後的 discordtune 4,919 列上，0c 的融合對解碼器是 $+49/-13$，對出貨的流程是 $+39/-110$（淨值 −71，比例 2.82）；出貨的流程對解碼器是 $+140/-33$。其中有 34 列 0c 讀不了；紀錄有這個列數，沒有寫這些列在融合那一臂怎麼計分。量的只有「0c 加解碼器」這一種融合；「0c 疊在出貨的重排之上」沒有量（§rl-071、§rl-072）。在這個候選池與這些融合設定之下，0c 在 v0.4.1 時於作者聊天上的優勢，到 v0.6.0 已經沒有了。

規則式轉換器。 Ruhi 是一個規則式的開源轉換器，在三個公開集合上跑，用它自己的讀音、沒有前文。整句全對：錯字回報 51/168，善解 131（聊天、有前文）與 127（沒前文）；dev302 154/302，善解 280 ／ 279；typing76 39/74，善解 72 ／ 72。只有 Ruhi 對的列：14、3、1；那 14 列裡有 4 列是「長」讀「ㄓㄤˇ」的列，後來由解碼器的讀音項修好（§rl-069）。

6 負面結果

表 6 把沒過的實驗依失敗的種類分類；完整的清單（含代號與紀錄裡的每個數字）在附錄 D。四種失敗能支持的結論不一樣。效果退步是在測過的條件下不利於那個想法的證據。證據不足是效果比實驗偵測得到的還小，不等於沒有效果。資源不可行對「預算更大時準不準」沒有說任何話。程序無效是結果不能照登記的方式解讀。登記沒過本身不等於那條線結案；結不結案是另外記錄的決定（§rl-052）。

表 6: 負面結果，依失敗的種類分。數字照紀錄（附錄 D）；沒有另外寫集合的「列」是作者聊天開發集上的淨列數。

實驗	對照它的條件，結果是
效果退步：方向反了	
裝置端基礎模型當判斷器	聊天開發集 83.5%→75.4%；調換候選順序會改掉它 49–55% 的選擇
更大的重排模型（3,020 萬參數）	−30 列；對 1,440 萬參數的模型 −97
listwise 訓練目標	對出貨的重排模型淨值 −38
看 63 個字前文的蒸餾學生	−39
口語表當解碼器的一項	整條流程 128/92：照當時的標準改壞太多
依讀音分開計 bigram	lytune 聊天 −1,457 列；抽查 157 個位置，讀音標錯 64 個
用交叉熵差挑新文字	網路用語檢查集書面 −35（$p < 0.0001$）
證據不足：比實驗偵測得到的還小	

實驗	對照它的條件，結果是
加口語文字重訓重排模型（第一次嘗試）	聊天判讀集上的估計淨值 +16，最小可偵測效果 +32
三元解碼器上更大的重排模型	以看到的效果大小，過不了登記的 Holm 檢定
64 字前文，重訓或接續訓練	U_2 0、−9、−32，種子雜訊 −6、−17：沒有方向
用沒轉換的文字從頭重訓重排模型	U_2 −52、−44、−46，在同配方重訓的流動之內
放寬的書面融合格（確認）	淨值 +19、$p = 0.0928$、比例 0.72
從 0c 模型蒸餾的學生	三個變體 −2、−2、+7（另外兩個 −35、−41）
資源不可行：延遲、記憶體或大小	
本機大型語言模型重排	p95 848 ms，上限 300 ms（正確率 91.4%，門檻 95%）
雲端大型語言模型判斷器	最好 92.7%、p95 3,146 ms；一秒內最好 92.4%
最大的重排模型	放不進 8 GB 的 GPU
不剪枝的三元表	1,133.3 MB，是唯一在它選出的值上通過的大小
程序無效：結果不能照登記解讀	
「改壞 1 句 ≈ 改對 10 句」的選值權重	4,091 列裡最多只能得到淨值 +17 到 +23，和換平台、換種子的雜訊一樣大
跳一個詞的詞類橋接（確認）	登記的臂不是先前量過的那一臂；0/2
在／再的詞態分類器	審查時發現融合混用兩種對數底、類別先驗失真；觸發停止條件
網路論壇與公版文字當評測文字	抽樣有 5.3–12.7% 的列不能用；過濾後仍有 13.8% 要改；放棄

負面結果說了什麼。 (i) 在測過的條件下（1,400 萬到 3,000 萬參數的字元模型、出貨格式裡的兩個字前文、第 8 節列的訓練文字），把神經部件做大、給它更長的前文、改用判別式訓練或從更強的教師蒸餾，都沒有通過。其中好幾個是「證據不足」而不是「效果退步」：它們和重訓的流動分不出來，還有一個在手上的 GPU 上跑不了。(ii) 選值用的權重與指標，用之前要先算它的天花板與雜訊。在「改壞 1 句 ≈ 改對 10 句」的權重下，1,440 萬參數的模型是改對 70、改壞 10，加權效用 $U_{10} =$ 改對 $- 10 \times$ 改壞 $= -30$，語意校正因此預設關著出貨；事後才算出，在這個權重下就算直接在判讀集上挑值，4,091 列裡最多也只有淨值 +17 到 +23，和兩次除了平台與種子完全相同的執行之間的差（32 列）一樣大。(iii) 在候選之間挑的判斷器，先被候選涵蓋與延遲卡住，還輪不到模型好壞：專案一開始，前 16 名候選含正解的比例是 94.7%；有量延遲的本機與雲端判斷器 p95 是 510 到 3,146 ms，上限是 300 ms。(iv) 讀音標註、詞態與資料挑選的規則，都輸給它們原本要改進的單純計數。

7 結論

善解的選字流程是傳統的做法：一個 n-gram 解碼器，加一個 1,465 萬參數、在分差閘門之後的重排模型。約六週的證據說明它的正確率從哪裡來。在作者聊天的開發集上，第一名正確率從 v0.4.0 的 84.89% 升到 v0.7.0 的 94.07%（同一批子句在書面設定下從 83.56% 到 93.71%），凍結的錯字回報 80 列從 17 列對到 70 列對；最大的一步是用論壇與聊天頻道文字重建 bigram 與重排模型的那一版。在真正的書面文字（wikitune）上，同一段期間書面設定從 75.52% 到 81.46%；v0.6.0 那一代模型在那裡少了 23 列（2266→2243，+78/−101，不顯著），v0.7.0 多了 60 列（2243→2303，+97/−37）。這些都是開發集的數字。確認性的證據是表 1 的 13 次單次確認，其中 8 次成立；v0.6.0 那一代模型不在它們確認過的改動之內（附錄 E）。在發版監測用的保留集上，v0.5.0→v0.6.0 讓口語半邊從 115 到 124（$p = 0.012$，那個半邊的第五次使用），新啟用的新聞半邊從 125 到 128（$p = 0.25$）；v0.6.1→v0.7.0 之後是 127 與 126（$p = 0.25$、0.63）。把神經部件做大、看更長前文、判別式訓練或從更強的教師蒸餾的嘗試，在測過的條件下不是沒過登記的標準，就是和重訓的流動分不出來。v0.7.0 系統的逐項拿掉是描述性的：拿掉重排模型（連口語項）在三個集合各淨少 117 到 136 列，三元表只在書面設定有差，標準檔的分差門檻最多只改兩列（表 4）。在按鍵路徑的精簡重播裡，解碼器的每鍵 p95 在 v0.5.0 是 2.06 ms、v0.6.0 是 2.23 ms（後者不是閒置窗）；在另一台機器、開跑時負載 1.9 的同一場裡，v0.6.1 是 1.91 ms、v0.7.0 是 1.93 ms。重排模型不在按

鍵路徑上；完整 App 的延遲沒有量。規則、確認的帳與偏離規則的地方都公開，供別人檢驗；證據的限制在第 8 節。下一步（§rl-115、§rl-052、§ip-003、§ip-006）：論壇文字抓完、轉換校對表到位後，把 bigram 與重排模型一起重建；把讀音項的 P(讀音 | 字) 估準；用上殼已經回傳的 64 字前文；出 App 內計數器，讓「離線就對」能在 App 裡量。

8 限制

只有一個使用者。 到 v0.6.1 為止決定出貨的聊天集（discordtune、chatclause、chatjudge-k）是作者自己的訊息：只收作者自己寫的子句。CE3 的集合讀指定的一個伺服器的公開頻道、作者自己的 Threads 與 Facebook 貼文，加上論壇 A 推文與兩個公開微網誌的小抽樣；cjudge-1 是作者自己的 2,880 列加論壇 A 的 2,759 列（§plan-014、§me-021、§rl-087；第 9 節）。方法明說這一點（「聊天優先、書面只守門」），集合也大（各約 5,000 個子句），但除了 cjudge-1 的論壇 A 半邊，這裡沒有任何東西顯示能轉移到其他使用者的語域。

調參集和訓練文字重疊。 cvtune 有 507 列（13.4%）、wikitune 有 671 列（23.7%）整句出現在字元模型的訓練文字裡。在這兩個集合上選的每個值（第一個重排模型凍結的 α, τ 與之後的格）都可能偏高；讓它們出貨的決定是在 discordtune 上做的，它不在訓練文字裡（§rl-018、§plan-014）。

錯字回報集是單邊的，也當過線索。 它只在句子出錯時收，裡面的詞當過補詞庫的線索；專案的規則禁止對當過線索的列評分，但凍結 80 列上 17→70 這條序列是「回報的錯有修好」的證據，不是對沒看過的打字能推廣的證據（§me-012）。

讀音是產生的。 每一個調參、開發與判讀集的讀音都由工具產生，單字取主要讀音。扣次要讀音的項因此在這些集合上扣不到正解；唯一有真實按鍵的集合（錯字回報，327 個單字）裡有 4 個字讀的是會被扣分的讀音，其中一個改壞了一列（§rl-055、§me-019）。按鍵層級的效果（打字速度、候選窗、學習存檔）只有真引擎重播工具量得到、離線 CLI 量不到；到目前為止兩者在每一次核對都逐列相同。

離線對上 App。 第一次量時 29% 的錯字回報離線是對的。依回報時的版本拆開，聊天「離線就對」的比例在 v0.2.1 以前回報的列是 41%、v0.3.0–v0.4.5 是 27%、v0.4.6–v0.4.7 是 11%（推論：這一桶有很大一部分是之後的版本已經改好的列，不是離線與 App 的落差；沒有用舊版重跑）。跨版本基準測試不模擬學習、非同步重排的時序與殼讀前文的方式，所以 App 裡語意校正的進步可能比表上小（記為推論）。App 內計數器與「回報這句」在規劃中、還沒出貨（§ip-002、§ip-003、§me-017）。

調參時看過的集合。 除了每份只用一次的判讀集，所有集合在選值時都被看過；基準測試頁寫明沒被看過的只有保留集。保留集是發版監測用的集合，它的摘要統計回報過好幾次（第 4.2 節）；每個半邊 150 列；換新集的三份之間，同一版差到 13 句（約 4%），所以保留集數字不跨份比 [47, 48]。

dev302。 302 列裡 299 列是專案或 LLM 寫的，它在真實聊天的結果上反向過兩次；它只擋書面設定，其餘照報（§me-017）。

統計機制。 幾個門檻在看過資料之後改過；每一次的日期在附錄 C 與 E。小模型的分數在不同 CPU 上差到 10^{-6} 量級，所以同一次比較的所有臂在同一種 CPU 上跑，2026-10-29 起確認與出貨標準的數字在兩台機器各量一次（§me-015）。

資料。　來源只寫類型與規模。bigram 的計數文字是中文維基百科（前 200,000 篇，權重 1）、口語池（Common Voice zh-TW 與 Tatoeba 的句子、合成口語句，權重 5）、g0v `#rand0m` 頻道的公開訊息（權重 5），以及台灣一個公開論壇的文章與推文（論壇 A；推文權重 5、內文權重 1）[50]；重排模型的訓練文字是同一批來源加上 OpenSubtitles v2018 繁體字幕（清理後 4,432,866 句、42,814,742 個漢字；第 3.5 節）。釋出的模型檔是計數與權重、不是句子；每一個出貨模型的發佈說明都寫明不含任何私人對話 [50]。論壇 A 的來源身分與原始文字不公開，也沒有任何從它抽出的評測集公開（第 9 節）。

範圍。　只有 macOS、只有繁體中文（台灣）、只有注音；核心 2026-11-06 在 Windows 原生編譯並通過測試，內建的微軟注音也量了當參考（175 個按鍵狀態格，兩輪 173 格逐像素相同），但沒有 Windows 產品（§rl-116、§rl-119）。

版本。　本文描述的系統是 v0.7.0。跨版本基準測試量它用的 commit，和打 tag 的 commit 在評測規格綁定到發版的路徑（建置檔、核心、CLI、資料、異體字表、殼與腳本）上沒有差異 [47, 51]。它的發版數字和先前每一版一樣，是開發集與監測用的數字。v0.6.1 之後的改動裡，重排模型的權重、融合格、解碼器裡的讀音項與句首單字的學習規則各有一次單次確認（表 1）；口語項句首的估法、同分讀音的規則、重排保護詞表與第三個預設開的詞包沒有（附錄 E）。表 4 的逐項拿掉只在一台機器上跑一次，沒有照確認的做法在第二台機器重跑；私人集合只公開彙總數字，所以 discordtune 那一欄（含兩個閘門列）沒辦法用公開的檔重算。

沒有和其他輸入法同一天的比較。　和既有輸入法的比較只有附錄 F 的第一天快照：用的集合後來成了回歸集，其中一套輸入法還帶著學習狀態。所以本文不主張善解選字比任何其他輸入法好。

9　倫理聲明

私人聊天評測集。　到 v0.6.1 為止選值與決定出貨的聊天集（discordtune、chatclause、chatjudge-k）用作者自己的訊息建：作者在 Discord 伺服器頻道、Threads 與 Facebook 寫的文字，不含私訊，含個資或金鑰的訊息整則不收（§plan-014）。CE3 的集合（2026-10-27 起）只讀指定的一個 Discord 伺服器的公開頻道，加上作者自己的 Threads、Facebook 貼文與論壇 A 推文、兩個公開微網誌的小抽樣；CE3 cjudge-1 是作者自己的文字 2,880 列加論壇 A 推文 2,759 列，兩部分各抽 10% 校對，關鍵字命中暱稱與稱呼的列拿掉（§me-021、§rl-087）。判讀集只從固定的月份區間（2015-07～2026-07）抽，2026-09 永遠排除，2026-10 以後用善解打的文字不當正解（§ip-003）。這些集合不公開；本文只報它們的大小與在它們上面算出的統計數字，專案的評測工具對私人集合也只印數字 [51]。

出貨的模型與私人文字。　出貨的模型只在公開文字上計數或訓練；沒有任何私人訊息進入釋出的檔案，每個模型的發佈說明都寫明這一點 [50]。評測規格記下這條規則的理由：群聊裡有其他人的發言，所以這類文字只能當評測或調參用，不能用來訓練會隨安裝檔散布的模型 [51]。

論壇文字。　台灣一個公開論壇（論壇 A）的文章與推文用於詞頻計數、訓練，以及不公開的 CE3 判讀集裡論壇 A 的那一部分（表 8）；本文只用這個代號稱呼它，原始文字不散布，也沒有任何從它抽出的評測集公開（[50]、§rl-087；第 8 節）。

新聞文字。　私人的新聞開發集與保留集的新聞半邊來自一個未授權再散布的新聞來源；檔案放在 repo 外，repo 只記列數與 SHA-256，而且從不用來訓練 [48]。

保留集的處理。 保留集只由獨立的 agent（沒參與實作的 LLM session）、只在推 tag 前量，只回摘要統計；每一份 executor 的 brief 都禁止讀保留集的檔 [48]。

參考文獻

[1] 0c-biggo. 0c-ime (agpl-3.0; model weights mit), 2026. URL https://github.com/0c-biggo/0c-ime. Accessed 2026-11-10.

[2] amzxyz. rime-wanxiang, 2026. URL https://github.com/amzxyz/rime-wanxiang. Accessed 2026-11-11.

[3] azooKey. Zenzai: neural kana-kanji conversion in AzooKeyKanaKanjiConverter, 2024. URL https://github.com/azooKey/AzooKeyKanaKanjiConverter/blob/main/Docs/zenzai.md.

[4] azooKey. AJIMEE-Bench, 2026. URL https://github.com/azooKey/AJIMEE-Bench. Accessed 2026-11-11.

[5] Taylor Berg-Kirkpatrick, David Burkett, and Dan Klein. An empirical investigation of statistical significance in NLP. In *Proceedings of the 2012 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning*, pages 995–1005, 2012. URL https://aclanthology.org/D12-1091/.

[6] Avrim Blum and Moritz Hardt. The Ladder: A reliable leaderboard for machine learning competitions. In *International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pages 1006–1014, 2015. URL https://proceedings.mlr.press/v37/blum15.html.

[7] Xavier Bouthillier, Pierre Delaunay, Mirko Bronzi, Assya Trofimov, Brennan Nichyporuk, Justin Szeto, Naz Sepah, Edward Raff, Kanika Madan, Vikram Voleti, Samira Ebrahimi Kahou, Vincent Michalski, Dmitriy Serdyuk, Tal Arbel, Chris Pal, Gaël Varoquaux, and Pascal Vincent. Accounting for variance in machine learning benchmarks. *arXiv preprint arXiv:2103.03098*, 2021. URL https://arxiv.org/abs/2103.03098.

[8] Peter F. Brown, Vincent J. Della Pietra, Peter V. deSouza, Jennifer C. Lai, and Robert L. Mercer. Class-based n-gram models of natural language. *Computational Linguistics*, 18(4): 467–479, 1992.

[9] BYVoid. OpenCC: Open chinese convert, 2026. URL https://github.com/BYVoid/OpenCC. Accessed 2026-11-11.

[10] Stanley F. Chen and Joshua Goodman. An empirical study of smoothing techniques for language modeling. *Computer Speech & Language*, 13(4):359–393, 1999. doi: 10.1006/csla.1999.0128.

[11] Zheng Chen and Kai-Fu Lee. A new statistical approach to Chinese Pinyin input. In *Proceedings of the 38th Annual Meeting of the Association for Computational Linguistics*, pages 241–247, 2000. URL https://aclanthology.org/P00-1031/.

[12] Thomas G. Dietterich. Approximate statistical tests for comparing supervised classification learning algorithms. *Neural Computation*, 10(7):1895–1923, 1998.

[13] Rotem Dror, Gili Baumer, Segev Shlomov, and Roi Reichart. The hitchhiker's guide to testing statistical significance in natural language processing. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1383–1392, 2018. URL https://aclanthology.org/P18-1128/.

[14] Cynthia Dwork, Vitaly Feldman, Moritz Hardt, Toniann Pitassi, Omer Reingold, and Aaron Roth. The reusable holdout: Preserving validity in adaptive data analysis. *Science*, 349(6248): 636–638, 2015. doi: 10.1126/science.aaa9375.

[15] Funmula / BigGo. 0c zhuyin input method: character-selection technology page and the MIT-licensed 0c-sequence-d512 model, 2026. URL https://0c.biggo.com/technology/. Accessed 2026-11-10.

[16] Jianfeng Gao, Joshua Goodman, Mingjing Li, and Kai-Fu Lee. Toward a unified approach to statistical language modeling for Chinese. *ACM Transactions on Asian Language Information Processing*, 1(1):3–33, 2002. doi: 10.1145/595576.595578.

[17] Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems 30*, 2017. URL https://arxiv.org/abs/1705.08500.

[18] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, pages 1321–1330, 2017. URL https://arxiv.org/abs/1706.04599.

[19] Lars Hellsten, Brian Roark, Prasoon Goyal, Cyril Allauzen, Françoise Beaufays, Tom Ouyang, Michael Riley, and David Rybach. Transliterated mobile keyboard input via weighted finite-state transducers. In *Proceedings of the 13th International Conference on Finite State Methods and Natural Language Processing (FSMNLP 2017)*, pages 10–19, 2017. URL https://aclanthology.org/W17-4002/.

[20] Yafang Huang, Zuchao Li, Zhuosheng Zhang, and Hai Zhao. Moon IME: Neural-based Chinese Pinyin aided input method with customizable association. In *Proceedings of ACL 2018, System Demonstrations*, pages 140–145, 2018. URL https://aclanthology.org/P18-4024/.

[21] Zhongye Jia and Hai Zhao. KySS 1.0: a framework for automatic evaluation of Chinese input method engines. In *Proceedings of the Sixth International Joint Conference on Natural Language Processing*, pages 1195–1201, 2013. URL https://aclanthology.org/I13-1170/.

[22] Zhongye Jia and Hai Zhao. A joint graph model for Pinyin-to-Chinese conversion with typo correction. In *Proceedings of the 52nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1512–1523, 2014. URL https://aclanthology.org/P14-1142/.

[23] Mike Tian-Jian Jiang, Cheng-Wei Lee, Chad Liu, Yung-Chun Chang, and Wen-Lian Hsu. Robustness analysis of adaptive Chinese input methods. In *Proceedings of the Workshop on Advances in Text Input Methods (WTIM 2011)*, pages 53–61, 2011. URL https://aclanthology.org/W11-3509/.

[24] Reinhard Kneser and Hermann Ney. Improved backing-off for M-gram language modeling. In *Proceedings of the IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, pages 181–184, 1995.

[25] Philipp Koehn. Statistical significance tests for machine translation evaluation. In *Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 388–395, 2004.

[26] Taku Kudo, Toshiyuki Hanaoka, Jun Mukai, Yusuke Tabata, and Hiroyuki Komatsu. Efficient dictionary and language model compression for input method editors. In *Proceedings of the Workshop on Advances in Text Input Methods (WTIM 2011)*, pages 19–25, 2011. URL https://aclanthology.org/W11-3503/.

[27] Roland Kuhn and Renato De Mori. A cache-based natural language model for speech recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 12(6):570–583, 1990.

[28] levelel. librime-witogram, 2026. URL https://github.com/levelel/librime-witogram. Accessed 2026-11-11.

[29] Pierre Lison and Jörg Tiedemann. OpenSubtitles2016: Extracting large parallel corpora from movie and TV subtitles. In *Proceedings of the 10th International Conference on Language Resources and Evaluation (LREC 2016)*, pages 923–929, 2016.

[30] Sven Martin, Jörg Liermann, and Hermann Ney. Algorithms for bigram and trigram word clustering. *Speech Communication*, 24(1):19–37, 1998. doi: 10.1016/S0167-6393(97)00062-9.

[31] metasequoiaime. pinyin-ime-reranker-4m (model card), 2026. URL https://huggingface.co/metasequoiaime/pinyin-ime-reranker-4M. Accessed 2026-11-11.

[32] Mahdi Milani Fard, Quentin Cormier, Kevin Canini, and Maya Gupta. Launch and iterate: Reducing prediction churn. In D. Lee, M. Sugiyama, U. Luxburg, I. Guyon, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 29. Curran Associates, Inc., 2016. URL https://proceedings.neurips.cc/paper_files/paper/2016/file/dc5c768b5dc76a084531934b34601977-Paper.pdf.

[33] Robert C. Moore and William Lewis. Intelligent selection of language model training data. In *Proceedings of the 48th Annual Meeting of the Association for Computational Linguistics (Short Papers)*, pages 220–224, 2010.

[34] Hussein Mozannar and David Sontag. Consistent estimators for learning to defer to an expert. In *Proceedings of the 37th International Conference on Machine Learning*, pages 7076–7087, 2020. URL https://arxiv.org/abs/2006.01862.

[35] Nanako0129. 善解 shanjie: an open-source macos zhuyin input method, 2026. URL https://github.com/Nanako0129/shanjie.

[36] Hermann Ney, Ute Essen, and Reinhard Kneser. On structuring probabilistic dependences in stochastic language modelling. *Computer Speech & Language*, 8(1):1–38, 1994. doi: 10.1006/csla.1994.1001.

[37] OpenVanilla. McBopomofo: Zhuyin input method for macos (MIT; phrase data from libtabe), 2026. URL https://github.com/openvanilla/McBopomofo. Accessed 2026-11-11.

[38] Benjamin Recht, Rebecca Roelofs, Ludwig Schmidt, and Vaishaal Shankar. Do ImageNet classifiers generalize to ImageNet? In *International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 5389–5400, 2019. URL https://proceedings.mlr.press/v97/recht19a.html.

[39] Nils Reimers and Iryna Gurevych. Reporting score distributions makes a difference: Performance study of LSTM-networks for sequence tagging. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pages 338–348, 2017. URL https://aclanthology.org/D17-1035/.

[40] RIME. librime: Rime input method engine, the core library, 2026. URL https://github.com/rime/librime. Accessed 2026-11-10.

[41] Hisami Suzuki and Jianfeng Gao. A comparative study on language model adaptation techniques using new evaluation metrics. In *Proceedings of Human Language Technology Conference and Conference on Empirical Methods in Natural Language Processing*, pages 265–272, 2005. URL https://aclanthology.org/H05-1034/.

[42] Minghuan Tan, Yong Dai, Duyu Tang, Zhangyin Feng, Guoping Huang, Jing Jiang, Jiwei Li, and Shuming Shi. Exploring and adapting Chinese GPT to Pinyin input method. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1899–1909, 2022. URL https://aclanthology.org/2022.acl-long.133/.

[43] Yee Whye Teh. A hierarchical Bayesian language model based on Pitman-Yor processes. In *Proceedings of the 21st International Conference on Computational Linguistics and 44th Annual Meeting of the ACL*, pages 985–992, 2006.

[44] The Chewing Project. libchewing: the intelligent phonetic input method library, 2026. URL https://codeberg.org/chewing/libchewing. Accessed 2026-11-10.

[45] Hiroyuki Tokunaga, Daisuke Okanohara, and Shinsuke Mori. Discriminative method for Japanese kana-kanji input method. In *Proceedings of the Workshop on Advances in Text Input Methods (WTIM 2011)*, pages 10–18, 2011. URL https://aclanthology.org/W11-3502/.

[46] Nanako Tsai. Shanjie v0.7.0 leave-one-out ablation: results and the offline recomputation of the two gate rows (`eval/ablation/v0.7.0/`), 2026. URL https://github.com/Nanako0129/shanjie/tree/e88f4a364d414780678999ee471096b40bb6212c/eval/ablation/v0.7.0. As of commit e88f4a36; the directory was added after the v0.7.0 tag.

[47] Nanako Tsai. Shanjie cross-version benchmark: result files (`eval/bench/`), 2026. URL https://github.com/Nanako0129/shanjie/tree/56979b2e2f6c7fd7a7288c38ed772d26f297e0e4/eval/bench. As of commit 56979b2e (the v0.7.0 tag). The public sets are shown at https://shanjie.nyanako.com/benchmark.

[48] Nanako Tsai. Shanjie evaluation sets: roles, hashes and hold-out usage record (`eval/ README.md`), 2026. URL https://github.com/Nanako0129/shanjie/blob/56979b2e2f6c7fd7a7288c38ed772d26f297e0e4/eval/README.md.

[49] Nanako Tsai. 善解研究摘錄：論文引用的開發紀錄段落, 2026. URL https://github.com/Nanako0129/shanjie/blob/5a33e0adbd09bb4885c53db132e978103030adf1/docs/research-excerpt.md. 以 commit 5a33e0ad（2026-11-11；在 v0.7.0 的 tag 之後，補了計畫的四段）為準。正文裡 §rl-012 這種形式的引用是這份檔案裡的錨點.

[50] Nanako Tsai. Shanjie release notes for the input method and the model files (`docs/ releases/`), 2026. URL https://github.com/Nanako0129/shanjie/tree/56979b2e2f6c7fd7a7288c38ed772d26f297e0e4/docs/releases.

[51] Nanako Tsai. Shanjie specifications (`docs/spec/`): decoder, models, semantic re-rank, learning, lexicon and packs, evaluation, privacy, 2026. URL https://github.com/Nanako0129/shanjie/tree/56979b2e2f6c7fd7a7288c38ed772d26f297e0e4/docs/spec.

[52] Emiel van Miltenburg, Chris van der Lee, and Emiel Krahmer. Preregistering NLP research. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 613–623, 2021. URL https://aclanthology.org/2021.naacl-main.51/.

[53] vChewing. vChewing (唯音) input method, 2026. URL https://github.com/vChewing/vChewing-macOS. Accessed 2026-11-11.

[54] Jinghui Xiao, Qun Liu, Xin Jiang, Yuanfeng Xiong, Haiteng Wu, and Zhe Zhang. PERT: A new solution to Pinyin to character conversion task. *arXiv preprint arXiv:2205.11737*, 2022. URL https://arxiv.org/abs/2205.11737.

[55] Yuqing Xie, Yi-An Lai, Yuanjun Xiong, Yi Zhang, and Stefano Soatto. Regression bugs are in your model! measuring, reducing and analyzing regressions in NLP model updates. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 6589–6602, 2021. URL https://aclanthology.org/2021.acl-long.515/.

[56] Sijie Yan, Yuanjun Xiong, Kaustav Kundu, Shuo Yang, Siqi Deng, Meng Wang, Wei Xia, and Stefano Soatto. Positive-congruent training: Towards regression-free model updates. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 14299–14308, 2021.

[57] Hui Zhang and David Chiang. Kneser-Ney smoothing on expected counts. In *Proceedings of the 52nd Annual Meeting of the Association for Computational Linguistics*, pages 765–774, 2014. URL https://aclanthology.org/P14-1072/.

[58] Zhuosheng Zhang, Yafang Huang, and Hai Zhao. Open vocabulary learning for neural Chinese Pinyin IME. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 1584–1594, 2019. URL https://aclanthology.org/P19-1154/.

A 詞庫、符號與學習規則

這份附錄補上第 3 節只做摘要的部分。

A.1 詞庫

基底是小麥注音的詞庫。上面疊中文維基百科與維基詞典標題建的疊加層（CC BY-SA 4.0；較嚴的篩法「V2t」去掉簡體專用字後留 342,761 詞，因為全收 104 萬標題反而讓選字變差——專名污染了候選；§rl-003）；依教育部規範產生的「一／不／法」變調讀音列（1,302 列，加輕聲容錯層後 1,502 列）；名單層（台灣的實況主、頻道、店家、企業、台灣以外的中文圈人名）；兩輪收的網路用語（507 與 910 詞）；以及三個預設開的可選詞包：動漫與遊戲詞包（Wikidata 來的 44,552 詞）、資訊＋ AI 詞包（國教院名詞、維基轉換組與手寫清單來的 1,016 詞），以及 v0.7.0 起的美妝與保養詞包（539 詞）[50]。疊加層的詞用上限先驗計分，分數不會高過它的語料頻率（式 (7)）。詞包建置工具標成強制的名字不走通語的統計，才不會被常用字拆掉（第 A.3 節）。

表 7: 符號。次數都是詞圖上的加權期望次數、四捨五入。

符號	值	意義
r、r_j		打進來的音節；一個讀音
w、v、u		詞庫條目：這個詞、前一個、前前一個
π、$S(\pi)$、$E(\pi)$		路徑、路徑分數、收尾（句尾）項
λ_p	0.5 ／ 0.7	模型對先驗的權重，聊天／書面
$lp(w)$、$lp'(w)$	$\log_{10}$	詞庫先驗；學習後的先驗
$s(r,w)$	$\log_{10}$	條目封頂前的原始檔案分數
$c(\cdot)$、$t(\cdot)$、N		bigram 或三元的次數；剪枝前的前文總次數；unigram 總次數
D、D_3	0.75	絕對折扣（bigram、三元）
$pb(w)$、$pb_{\mathrm{KN}}(w)$		unigram 回退值；它的 Kneser–Ney 替換值
θ、$N_\theta(w)$、β_p	1、—、1 ／ 0.75	接續門檻與接續次數；KN 權重，聊天／書面
$C(w)$、K、μ、$\varepsilon_{\mathrm{cls}}$	512、0.2、0.1	詞類、類別數、詞類權重、類別平滑
$e(w)$、$q(w \mid v)$		類別發射；詞類平滑的回退
T_p、F	0.5	該設定的三元表；三元下限
$\eta(w)$、η_{dem}、η_{sound}、η_{adj}	2.5、2.0	降權與容錯扣分
$\rho(w)$、$\xi(r,w)$、λ_r、$c_{\max}$	2.0、4	讀音項、它的對數比、權重、截斷
y_i、n_c、ng_i	$n_c \le 8$	重排候選、候選數、它們的解碼器分數
k、ll_i、$\tilde{ll}_i$	$\lvert k\rvert \le 2$	前文鍵；字元模型分數；常談融合後的值
(α,τ,δ)	依強度	融合格：融合權重、解碼器分差門檻、替換分差
f_i、U		候選 i 的融合分數；可用的候選
P_{CL}、lc_i、col_i、o_i、κ	0.5	常談表與它的路徑分數、扣分後的分數、詞包詞數、每詞扣分
μ_{CL}、T_g、g_i、γ	0.5、1.0、—、0.25	常談權重、閘門溫度、閘門、每個詞包詞的權重
ω、ε_ℓ	6.0	學習紀錄的權重（衰減後）；學習加分
b_i、ℓ_i、$\varepsilon_{\mathrm{fine/mid/coarse}}$	4、4、2	句首紀錄的份量、學習項、三層的權重
U_2		改對 $-2\times$ 改壞，選重排器訓練用的分數

A.2 符號

表 7 列出用在一個以上小節的每個符號；同一個符號全文只代表一件事。詞庫條目是（讀音，字串）對；我們用 w 代表條目、讀音隱含在內，所以 $lp(w)$、$\eta(w)$、$\rho(w)$ 都是條目的函數（同一字串在另一個讀音下是另一個條目），只在對讀音本身運算的地方才寫 (r,w)。

A.3 詞庫先驗、詞包、降權與容錯

每個詞庫條目 (r,w) 有一個檔案分數 $s(r,w)$（$\log_{10}$）：基底列是小麥注音的分數，疊加層列是依詞長給的常數。令 $c(w)$ 是模型裡 w 的 unigram 次數、N 是加權 unigram 總次數。一個詞算不算「在疊加層」看字串：疊加層檔列到的詞，它每一個條目都封頂，同字串的基底列也包括在內。解碼器用的先驗是

$$lp(w) = \begin{cases} s(r,w) & \text{不在疊加層；強制或模型沒見過的詞包列；} \\ \min\big(s(r,w), \log_{10}(c(w)/N)\big) & \text{其他疊加層的詞，} c(w) > 0; \\ s(r,w) - 1.0 & \text{其他疊加層的詞，} c(w) = 0, \end{cases} \tag{7}$$

所以疊加層的詞分數不會高過它的語料頻率。詞包建置工具標成強制的詞包列（同讀音上要贏過常用字寫法的名字）不論 $c(w)$ 多少都保留 $s(r,w)$，並且當成模型不認得的詞計分：$P(w \mid v) =$

$\mathrm{back}(v)\,10^{lp(w)}$、沒有詞類項，下一個詞也當前文未知計分。降權表列出辭典沒有的讀音（與敏感詞）的 $(r, w, \eta_{\mathrm{dem}})$，$\eta_{\mathrm{dem}}$ 是式 (1) 的 $\eta(w)$ 的一部分。容錯讀音也加進 $\eta(w)$：條目讀音裡有一個音節是經模糊音對（ㄣ／ㄥ、ㄓ／ㄗ、ㄔ／ㄘ、ㄕ／ㄙ；預設關）換到的，加 $\eta_{\mathrm{sound}} = 2.5$；詞庫沒有的音節經鄰鍵讀到的，加 $\eta_{\mathrm{adj}} = 2.0$。

A.4 Kneser–Ney 接續分布側檔

側檔對每個詞彙內的詞 w 存接續次數 $N_\theta(w)$：四捨五入後 $c(v, w) \geq \theta$ 的不同前文 v 的個數，$\theta = 1$；同一個詞的異體寫法算成一類、只算一次，N_θ 與下面的分母都是。回退值變成

$$pb_{\mathrm{KN}}(w) = \beta_p \,\frac{N'(w)}{\sum_{w''} N'(w'')} + (1 - \beta_p)\,10^{lp(w)}, \qquad N'(w) = N_\theta(w) + 1, \tag{8}$$

對詞彙內的詞取代式 (2) 每一個分支裡的 $pb(w)$；詞彙外的詞照舊用 10^{lp}，而 $\langle/s\rangle$ 沒有詞庫先驗，有沒有側檔都用它的句尾比率 eos/N 當回退值。$\beta_{\text{聊天}}$ 是檔頭的值（1），$\beta_{\text{書面}} = 0.75$。+1 讓 $pb_{\mathrm{KN}} > 0$，而且固定 lp 時對 N_θ 單調。計數是四捨五入的期望次數，所以這是接續計數的「門檻＋整數」近似，不是 [57] 的分數次數版本；折扣 D 也不重估（§rl-017）。

A.5 詞類回退

詞類檔給最常見 40,000 個詞每個一個類別 $C(w)$（Brown 式 exchange 分群，$K = 512$ 類；表有 $K + 3$ 格，$\langle s\rangle$ 與 $\langle/s\rangle$ 各有自己的類別）、一張平滑 $\varepsilon_{\mathrm{cls}} = 0.1$ 的類別 bigram 表 $P_C(d \mid c)$，以及發射 $e(w) = \mathrm{uni}(w)/\sum_{w' \in C(w)} \mathrm{uni}(w')$，$\mathrm{uni}(w)$ 是分群語料裡該詞的 unigram 次數（不是模型的 $c(w)$）。bigram 沒有保留條目、而且兩個詞都有類別時，

$$q(w \mid v) = (1 - \mu)\,pb(w) + \mu\,P_C\big(C(w) \mid C(v)\big)\,e(w), \qquad \mu = 0.2, \tag{9}$$

側檔開著時這裡的 pb 已經是 Kneser–Ney 的值；任一詞沒有類別時 $q(w \mid v) = pb(w)$。所以詞類項從不碰保留的 bigram，只重新分配回退質量。權重引進時 $\mu = 0.8$，v0.4.6 起 0.2（附錄 C；§rl-014、§rl-030）。重建時計數不變的詞沿用類別，只替新詞分類。

A.6 三元表

三元表（書面 7,700,964 條；聊天 950,000 條）對每個存著的前文 (u, v) 存保留後接詞的四捨五入次數 $c_3(u, v, w)$ 與剪枝前總次數 $t_3(u, v)$，折扣 $D_3 = 0.75$。令 $P_2(w \mid v)$ 是式 (2) 的 bigram（用該設定的 β_p），

$$\begin{aligned} P_3(w \mid u, v) &= \max\Big(\frac{\max(c_3(u, v, w) - D_3, 0)}{t_3(u, v)} + \mathrm{back}_3(u, v)\,P_2(w \mid v),\; F\,P_2(w \mid v)\Big), \\ \mathrm{back}_3(u, v) &= 1 - \sum_{w' :\, c_3(u, v, w') > 0} \frac{c_3(u, v, w') - D_3}{t_3(u, v)}, \end{aligned} \tag{10}$$

下限 $F = 0.5$；(u, v) 沒存、或 u 未知時 $P_3 = P_2$。句首的歷史是 $(\langle s\rangle, \langle s\rangle)$，所以第一個詞照樣查表；$u$ 未知的情況是這一段接在一個固定詞之後：引擎只傳那一個左邊的詞，所以固定詞之後的第一個詞用 bigram 計分，固定詞之後只有一個詞的段收尾也用 bigram。該設定有表時，式 (1) 的詞項用 P_3；收尾項用 $P_3(\langle/s\rangle \mid u, v)$，$pb$ 是句尾比率。表以唯讀記憶體映射讀，頁面是乾淨的檔案支援頁、不計入程序 footprint；下限 F 是解碼器的常數，不在表檔裡（§rl-032、§rl-037）。

A.7　解碼器裡的讀音項

評測的讀音與詞庫的佔位分數一直讓常用單字的罕用讀音贏：「否則」之後打「變快」出成「便會」(§rl-081)。對在音節 r 下使用的單字條目 w，令 $s(r,w)$ 是原始檔案分數、$s^{*}(w)=\max_{r'} s(r',w)$ 是它最好的讀音。定義（取第一個符合的分支）

$$\xi(r,w)=\begin{cases}0 & (r,w)\in\mathcal{X},\\ \log_{10} M(r,w)-\max_{r'}\log_{10} M(r',w) & s(r,w)=-6.8\ \text{(底分)},\\ 0 & (r,w)\in\mathcal{S},\\ s(r,w)-s^{*}(w) & \text{其他},\end{cases}\tag{11}$$

$$\rho(w)=\lambda_r\max\big(\xi(r,w),-c_{\max}\big),$$

$\lambda_r=2.0$、$c_{\max}=4$；$\mathcal{S}=\{(\text{ㄧˊ},\text{一}),(\text{ㄧˋ},\text{一}),(\text{ㄅㄨˊ},\text{不})\}$ 是變調讀音；$\mathcal{X}$ 是詞庫把辭典第一讀音排在第二的 12 對（朴、嗎、率、哩、還、和、癌、賃、波、相、嚇、教）；$M(r,w)=10^{s(r,w)}+\sum 10^{s(\text{詞})}$，加總的是基底裡每一個在該位置含 w、讀 r 的多字詞（底分對的「mc」估法；變調對若剛好在底分會取 mc 值，出貨詞庫裡沒有這種情況）。多字條目與被學習抬起的詞 $\rho(w)=0$（這一項最多扣 $\lambda_r c_{\max}=8$、學習最多加 6，使用者教過的讀音要站得住）。這一項在路徑分數裡、beam 照它剪枝，兩種設定、三檔、語意校正關都生效；它取代了早先放在重排融合裡的讀音項，也就是 v0.6.1 量到的那個形式（附錄 C）。

v0.7.0 多一種情況，在免扣之後套用。一個字至少有兩個單字讀音、這些讀音的 ξ 都是 0、而且不是只差聲調時，令 $m(r)$ 是基底詞庫裡含這個字、讀 r 的多字詞的 $10^{s(\text{詞})}$ 之和；$\log_{10}\big((m(r)+10^{-9})/(\max_{r'} m(r')+10^{-9})\big)\le -1$ 的讀音就用這個值當 ξ，其他維持 0。出貨詞庫裡有 132 對（讀音，字）因此分出高低，例如詞庫給同分的「莎」的兩個讀音；在公開的開發集、打字測驗與錯字回報（兩種設定）上，它只改了一列，是改對 [50, 51]。

A.8　改選學習

一筆紀錄是（前文鍵，讀音，詞）加權重 ω，每 14 天減半：$\omega_{\text{今天}}=\omega\cdot 0.5^{\max(\text{今天}-\text{當天},\,0)/14}$（時鐘倒退不會讓它變大）；低於 0.5 的紀錄不生效，低於 0.05 的刪掉。下面的 ω 都是衰減後的權重。前文鍵是詞之前最多兩個漢字（句首用一個哨兵，鍵再帶下一個音節）；單字只在剛好兩個字的前文完全相同、或句首鍵下才學。目前鍵下有生效紀錄的條目，

$$lp'(w)=\max\Big(lp(w),\ lp^{*}(r)+\varepsilon_\ell\,\frac{\omega}{\omega+1}\Big),\qquad \varepsilon_\ell=6.0,\tag{12}$$

$lp^{*}(r)$ 是讀音 r 下最高的封頂先驗；全域層（在兩種前文學到的詞傳到第三種）有自己的加分 $\varepsilon_{\text{global}}=0$，所以 v0.3.0 起等於關閉（表 10）。被抬起的條目用 $pb=10^{lp'}$ 計分，不經 Kneser–Ney 替換、也不經三元表（兩者都會把抬升縮回 $(1-\lambda)lp'$ 那一份），也不加讀音項。改選走時舊詞的紀錄減半，所以權重大的紀錄勝過權重小的。連續單字改選拼出的 2–4 字串學成新詞，先驗 $lp_{\text{新}}=-3+2\cdot\frac{\omega}{\omega+1}$，當模型不認得的詞計分。

句首單字（SINIT 片）在建候選池的那次解碼裡不抬升，改進重排融合。教過的字可能根本不在池子的 n_c 條路徑裡，所以引擎再解碼一次：把教過的字抬到該讀音最高的先驗，從結果取以那個字開頭的最佳路徑，還原成沒抬升的分數與先驗，池子裡還沒有同樣文字時放進池子的最後一格（永遠不是第一格）。一段的第一個詞是單字時，令 $b_i=\frac{\omega}{\omega+1}$ 是候選 i 第一個字在最長命中鍵層下的紀錄（細鍵：哨兵加下三個音節，組字在第二個音節就結束時第三格換成結尾標記；中鍵：兩個；粗鍵：一個）。式 (4) 的學習項是

$$\ell_i=\begin{cases}\varepsilon_{\text{fine}}\,(b_i-b_0) & \text{細鍵，}\varepsilon_{\text{fine}}=4,\\ \varepsilon_{\text{mid}}\max(b_i-b_0,0)\,\mathbb{1}[ll_i>ll_0] & \text{中鍵，}\varepsilon_{\text{mid}}=4,\\ \varepsilon_{\text{coarse}}\max(b_i-b_0,0)\,\mathbb{1}[ll_i>ll_0] & \text{粗鍵，}\varepsilon_{\text{coarse}}=2,\end{cases}\tag{13}$$

$\mathbb{1}[\cdot]$ 是指示函數，三層不相加。細鍵正負兩向（學到的字是錨點時，別的候選要贏它也得多付這一項）；中鍵與粗鍵只抬不壓，而且只在字元模型已經同向偏時。解碼器有把握時（$ng_0 - ng_1 \geq \tau$），只有 $\ell_i > \ell_0$ 的候選能換掉第一名，照一般的 δ 贏。這條規則在兩版沒過之後於 2026-11-03 通過預先登記的確認（第 C.4 節；§rl-094）。

B 評測集

表 8 列出各評測集的大小、讀音的來源與角色（第 4.2 節）。

表 8: 評測集。私人集合只寫大小。「讀音」欄寫讀音是真人打的還是專案工具從正解產生的。

集合	語域	列數	讀音	角色／來源
dev302	兩種	302	確認過	回歸（2026-10-20 起只擋書面）；299 列是專案或 LLM 寫的
typing76	兩種	76	確認過	回歸；專案自寫的打字測驗，CC0
probe（s2r）	兩種	83	產生	回歸；變調讀音變體
fw（功能字）	兩種	186	產生、校對	回歸；在／再、的／得／地、做／作
錯字回報	兩種	187（基準凍結前 80）	實際打的	回歸；真實錯字，CC0；單邊（只在出錯時收）
wikitypo	兩種	155	產生	回歸
cvtune ／ cvtune-clean ／-tw0	聊天	3,791 ／ 3,286 ／ 2,992	產生	調參；Common Voice zh-TW ＋ Tatoeba 的 20%
wikitune ／-clean ／-tw0	書面	2,827 ／ 2,181 ／ 2,182	產生	調參；維基第 300,000–304,999 篇
lytune	書面	4,811	產生	開發，只警示；立法院發言
discordtune（`cdev`）	聊天	4,958（過濾後 4,919）	產生	開發；作者的聊天，私人
chatclause（`ctune`）	聊天	5,761（過濾後 5,631）	產生	調參；私人
chatjudge-2	聊天	5,783	產生	判讀，用過一次；私人
CE3 cjudge-1	聊天	5,639（自己 2,880 ＋論壇 A 2,759）	產生	判讀，用過一次；私人
wikijudge-k	書面	每份 2,816–2,857	產生	判讀，一份一個決定
newstune	書面	2,000	產生	開發；私人新聞（維基新聞 72 ＋ 1,928）
保留集口語半邊（shard-1）	聊天	150	產生，抽查 40	發版監測（到 v0.7.0 用了 8 次，之後換下一份）；公開口語文字
保留集新聞半邊（news-1）	書面	150	產生	發版監測（到 v0.7.0 用了 4 次）；私人（維基新聞 30 ＋ 120）

C　各部件的沿革

這份附錄照建置的先後保留每個部件的紀錄，含專案的代號與日期，以及紀錄裡描述性的 p。附錄沿用專案紀錄的用語：「片」是一個實驗，「一發」確認就是單次確認，「守門」是把關用的檢查。表 1 第 1–13 列對應這裡的實驗：rerank-formal（第 1、2 列）、P4d（3）、DECIDE-1（4）、BRIDGE-2（5）、R1g（6）、SINIT 修訂七、八、九（7–9）、SR-TW-Q（10）、SR-TW-R（11）、FUSE-1（12）、FUSE-2（13）。模型以功能命名，帶代號、版本與規模（附錄 H）。

C.1　基準與前幾週

用小麥基底詞庫與 unigram 解碼器，302 列開發集 51.7% 對（寬鬆 52.6%；oracle@32 96.7%），原本的 227 列保留集 41.4%（44.5%；95.6%）[48]。加上維基／維基詞典疊加層，整體 oracle@64 從 96.9% 升到 97.8%。第一個 bigram（維基 50,000 篇、$\lambda = 0.5$、口語語料 $\times 5$）把開發集從 163 句對帶到 231 句，私人聊天錯誤率從 23.3% 降到 18.9%。之後的配對檢定機制下，統一先驗的 interp 加句尾項在 cvtune 淨修好 +112 列、wikitune +207（$p = 10^{-16}$、10^{-31}）；維基從 50,000 篇加到 200,000 篇多 +29 與 +71（$p = 0.004$、3×10^{-8}）；疊加層上限在 wikitune 修好 11、弄壞 0（$p = 0.001$）。這些調參集的進步第一次查看時沒有一個轉移到私人聊天驗證集（修好 32、弄壞 32），因此才收了 4,958 個子句的私人聊天調參集：在它上面第 4 輪最佳是 4,066，舊的最佳 4,045（修好 207、弄壞 186，$p = 0.31$）。892 個錯句裡 865 句是排序錯、27 句正解不在前 64 名；最常見的字對是再 → 在（31 句）、吃 → 喫（29）、做 → 作（20）、錢 → 前（18）、在 → 再（17）、甚 → 什（16）（§rl-005、§rl-007、§rl-008、§rl-009、§rl-010、§rl-012）。

bigram 在 v0.1.0（2026-10-04）出貨；那時的保留集聊天 175/227（77.1%）、書面 181/227（79.7%）、oracle@64 226/227（99.6%），達到涵蓋目標；模型 80,040,411 bytes，載入後 RSS 238 MB，每鍵 p95 1.18 ms。變調讀音變體在 cvtune 探針 +36（$p = 5.6 \times 10^{-9}$）、discordtune 探針 +62（$p = 6 \times 10^{-17}$），原讀音不退步。清掉詞庫與語料的簡體殘留（疊加層 3,489 列；七個殘留字 125,635→0 次）讓 cvtune 聊天 +15（$p = 0.02$）（§plan-008、§plan-010）。

C.2　出貨標準的沿革

第 4 節那套標準的歷史本身就是結果。第 1 條 2026-10-19 是「不顯著變差且改壞不多於出貨值」。2026-10-20 一個片在 dev302 聊天少了 7 列之後，dev302 聊天改成只報、加上 cvtune-clean 非劣性；邊界訂 −1.0 個百分點 而不是 −0.3 個百分點，因為常態近似估一個真正中性的改動在 −0.3 個百分點 約八成、在 −1.0 個百分點 約兩成會被擋下。2026-10-20 看過 P4b 之後改壞數換成「改壞 ≤ 26 或比例 ≤ 0.5」；同一晚看過 P4d（0.58）之後比例放寬到 ≤ 0.6。2026-10-27 換聊天集時拿掉「改壞 ≤ 26」的絕對數（§rl-047、§rl-049、§rl-053、§me-021）。看過數字之後才放寬的兩次都照實記下，只在放寬後的規則下過關的片在 chatjudge-2 確認過。

C.3　跨版本的細節

書面設定的 discordtune：4143（v0.4.0）→ 4172（v0.4.1，+30/−1）→ 4191（v0.4.3）→ 4177（v0.4.4）→ 4178（v0.4.6）→ 4218（v0.4.7，+138/−98，$p = 0.011$）→ 4267（v0.5.0，+69/−20）→ 4588（v0.6.0，+396/−75，$p = 9.6 \times 10^{-54}$）→ 4646（v0.7.0，+100/−42，$p = 1.2 \times 10^{-6}$）；聊天設定 v0.7.0 是 +31/−12（$p = 0.0054$）。v0.6.0 這一代模型在維基調參集上沒有變好：wikitune 聊天 2184→2160（+80/−104，$p = 0.0897$）、書面 2266→2243（+78/−101，$p = 0.0998$），都不顯著。資訊詞包檢查集在重建的階段 1（新的 bigram 配上一代的重排模型；發佈說明引的是這一組）是聊天 46→36、書面 39→27，出貨組合的定案判讀在這個檢查集上是聊天 3/14、書面 4/10，三檔都是負的（§rl-059、§rl-068；[50]）。v0.7.0 在維基調參集上升：聊天 2160→2186（+46/−20，$p = 0.0019$）、書面 2243→2303（+97/−37，$p = 2.2 \times 10^{-7}$）；dev302 書面（271→280，+10/−1，$p = 0.012$）與凍結的錯字回報書面（61→69，+9/−1，$p = 0.021$）也上升；lytune 聊天設定下降

(4430→4401，+33/−62，$p = 0.0038$；書面 4384→4403，+114/−95，$p = 0.21$)，維基錯字集書面下降 (137→131，+1/−7，$p = 0.070$)。v0.4.4 在立法院發言集 lytune 顯著退步 (聊天 4055→4010，+40/−85，$p = 7.0 \times 10^{-5}$；書面 4021→3979，+54/−96，$p = 0.00076$)；歸因與修法在第 C.4 節。

表 9: 各版對上一版的改對／改壞，只列有改到某個集合的版本（集合與設定同表 2；「—」表示那個集合逐列相同，或這裡用的來源沒有它的配對列數)。

版本	dev302	typing76	錯字回報	cvtune	discordtune (聊天)	discordtune (書面)
v0.4.1	18/0	3/0	9/1	84/3	114/24	30/1
v0.4.3	1/1	—	—	6/2	20/5	28/9
v0.4.4	3/3	1/0	2/3	19/18	40/51	72/86
v0.4.6	6/3	2/0	5/1	18/16	50/49	86/85
v0.4.7	—	—	—	—	—	138/98
v0.5.0	9/2	—	7/2	53/16	88/34	69/20
v0.6.0	14/3	2/0	38/1	95/50	336/49	396/75
v0.7.0	4/2	—	—	21/15	31/12	100/42

C.4 各部件

詞類（S2k)。 在 model-v3 上類別項讓 cvtune-native 聊天 +21、書面 +26（$p = 0.002$)，其他集合的變化不顯著，保留集 +2 ／ 0；出貨引擎的每鍵 p95 3.597 ms（負載低於核心數)。改變的列多半是「同音、前後文沒出現過」(期中／其中、戰機／戰績)，弄壞的集中在再／在與同音動詞 (接上／街上)；我們把這讀成和 bigram 加類別只看相鄰兩詞一致（推論，沒有另外驗證；§rl-014、§rl-015)。

Kneser–Ney 側檔。 選出的 $\theta = 1, \beta = 1$ 在四格調參集（13,236 列，0.25%）淨修好 +33 列；唯一顯著的一格是 wikitune 書面 2,131→2,149（+38/−20，$p = 0.025$)；保留集聊天 180→182 (+6/−4，$p = 0.75$)、書面 184→183（+5/−6，$p = 1.0$)；五個固定探針句一個都沒修到。它併進 model-v6 出貨（§rl-017)。

小型字元語言模型（S7a、S7c)。 在同樣 255,373,634 個加權 token（維基 240,750,484 字、口語 2,924,630 字 ×5）上訓了兩個大小：S（566 萬參數，最終損失 4.190 nats/token）與 M（1,440 萬，3.993)。用公開調參集選定、凍結的 (α, τ)，M 在 discordtune 聊天過了登記的關卡：4,958 列裡 +67 列（+1.35 個百分點，門檻 +1.0)，$p = 0.0008$，95% 區間 $[29, 105]$，ΔCER −0.23 個百分點；S 沒過（+31，$p = 0.149$)。公開集合的進步遠大於作者的聊天（dev302 聊天 241→277、錯字回報 15→31)；我們把這歸因於解碼器在聊天上已經 84.9% 對、剩下的錯比較難（沒有另外驗證)。int8 匯出（14.6 MB）在 Rust 重現 float32 的結果（+66，$p = 0.0010$)，保留集聊天 180→200/227 ($p = 0.0003$)。接進引擎讓每鍵 p95 多 0.16 ms（約 2.6 ms)，已顯示的字之後被改寫的比例從 14.5%（只用解碼器）降到 9.3%，2.2% 的鍵有閃回（§rl-018、§rl-019、§plan-014)。

在作者的語域上選值（E2，v0.4.1)。 在公開書面集合上調參兩次選出不會轉移的值之後，用作者自己的訊息建了聊天調參／判讀集，判讀期定為 2026-08（6,559 列)。先前出貨的保守值（在「改壞 1 句 ≈ 改對 10 句」的權重下選的）改對 82、改壞 7、淨值 +75、改壞率 0.11%；重選的 $(\alpha, \tau, \delta) = (0.3, 2, 0.25)$ 是 125/16、淨值 +109、0.24%（上界 0.37%)，隨 v0.4.1 出貨、語意校正預設開。基準測試上 v0.4.1 在 discordtune 改對 114、改壞 24（$p = 3.2 \times 10^{-15}$)，錯字回報 80 列從 17 升到 25（§plan-014)。

書面設定的三元表（v0.4.7）。 剪枝後的表（7,700,964 條、4,101,808 個前文）重對應到 model-v6、關 Kneser–Ney，在 Python 參考實作上調參集 +64（205/141，$p = 0.0007$）。接上出貨重排器之後，書面設定在 wikitune 改對 108、改壞 61（$p = 0.0004$），discordtune 164/123（$p = 0.018$），lytune 132/97（$p = 0.024$），沒有集合顯著變差。表多 136 MB 的 RSS（423 對 287 MB）與 0.01 ms 的 p95（2.15 對 2.13 ms）。v0.4.7 的基準測試：書面 wikitune 2183→2232（+101/−52，$p = 9.2 \times 10^{-5}$）、cvtune 3416→3440（+75/−51，$p = 0.04$）、lytune 4044→4080（+117/−81，$p = 0.0127$）、discordtune 4178→4218（+138/−98，$p = 0.011$）；聊天逐列相同（§rl-031、§rl-033、§rl-035）。

詞類權重與 lytune 退步（v0.4.6）。 lytune 上的因子歸因把 v0.4.3→v0.4.4 的退步（聊天 −45、書面 −42）拆成幾個階段。模型重建本身沒有差（3/3、7/7 互換）；詞類項單獨與 Kneser–Ney 單獨都小、不顯著；退步是兩者的交互作用：都開時詞類項的代價是 −39（88/127，$p = 0.009$）與 −56（103/159，$p = 0.0007$）。在出貨設定下掃 $\mu \in \{0, 0.2, 0.4, 0.6\}$，照登記的規則 $\mu = 0.2$ 最好：lytune 聊天 +27（100/73，$p = 0.048$）、書面 +65（130/65，$p < 0.0001$），discordtune 與錯字回報在雜訊內不變。v0.4.6 出貨 $\mu = 0.2$，書面的退步全數補回、聊天約六成（§rl-028、§rl-029、§rl-030）。

書面重排強度（rerank-formal，SJ-SR-6-14M）。 在三元解碼器上重選書面 (α, τ, δ)：在調參資料上以 0.5% 改壞率上限選出的標準格，在沒看過的維基集 wikijudge 主判讀過（$+76/-21$，$p < 0.0001$），卻在其他集合的聯集上沒過改壞率守門（229/13,292 = 1.72%），lytune 2.81%；不匯出。第二輪在 0.5% 到 0.1% 的上限各選一格，0.2% 那格（0.7, 2, 1）在看過的資料上以揭露的放寬（0.6%）進確認，和保守格一起在新的 wikijudge2（2,818 列）確認：$+39/-11$、淨值 +28、$p = 0.0001$、改壞率 0.35%；以及 $+20/-5$、$p = 0.004$、0.14%。新的書面標準 $(0.7, 2, 1)$ 成為 SJ-SR-6-14M；保留集對解碼器 $+12/-0$（$p = 0.0005$）、對前一個檔 $+8/-0$（$p = 0.008$）（§rl-039–§rl-043）。

常談表（P4 系列，v0.5.0）。 口語 bigram 當獨立的解碼器項（P4）在 discordtune 的 n-gram 層改對 168、改壞 89（+79，$p < 0.0001$），但整條管線重新評分（P4b）之後在那裡改壞 92 列（128/92，$p = 0.018$）、錯字回報改壞 6——在當時的標準下太多。把這一項搬進重排融合、以解碼器第一名當錨點、加軟閘門（P4d）：discordtune 74/43（淨值 +31，$p = 0.0053$，比例 0.58）、cvtune-clean 45/17（下界 +0.46 個百分點）、錯字回報 17/4，chatjudge-2 一發確認通過：108/39，淨值 +69，$p = 1.1 \times 10^{-8}$，比例 0.36，CER 2.85%→2.61%。只把 α 調到 0.5、不加表的對照臂在 chatjudge-2 是 61/38（+23）、discordtune 44/52；我們把進步歸給表、而不是調強的融合（從對照臂推論，沒有另外驗證）。疊在上面的讀音通道項（P5）在 discordtune 改對 23、改壞 0（$p = 2.4 \times 10^{-7}$），cvtune-clean 9/0，錯字回報 2/1；唯一改壞的是「長歪了」，「長」讀ㄓㄤˇ，詞庫給 −6.8 底分。兩者隨 v0.5.0 出貨：discordtune 聊天 $+88/-34$（$p = 1.1 \times 10^{-6}$）、lytune 聊天 $+191/-27$、cvtune $+53/-16$、書面 wikitune $+39/-5$（$p = 1.4 \times 10^{-7}$），保留集書面半邊 $+8/-0$（$p = 0.0078$）（§rl-046、§rl-048、§rl-053、§rl-054）。

SJ-LM-7 這一代模型（v0.6.0）。 用 g0v 與論壇 A 文字、不套 `TWPhrases` 重建 bigram，重對應三元表、重分群、重算常談、重訓重排器，分五個登記的階段。階段 1 比較計不計詞包詞（臂 A、B）在剪枝門檻 2 與 6：四臂都過第 1、3、4 條（B6：discordtune 4,366→4,534，$+201/-33$；錯字回報聊天／書面 58/51→95/99），四臂都不過第 2 條（dev302 書面改壞 6 列：祝你 → 助你、嚐嚐看 → 常常看、致病 → 治病、再講 → 在講……）；來源次數是「在講」論壇 A 推文 2,340 次對「再講」418 次（我們推論論壇 A 推文把在／再的口語寫法帶進同一個 bigram，沒有另外驗證；§rl-059）。這六列以記錄在案的第 2 條例外出貨（§rl-061；[50]）。門檻 3、4、5、8 各剩 6、6、5、6 列改壞，重選的書面融合格 6–8 列（§rl-060）。新來源用 Moore–Lewis 挑一半（ML6）比全收

(B6) 差：discordtune 4,534→4,523（14/25，$p = 0.108$），網路用語第二輪書面 -35（$p < 0.0001$）；留 B6。重算常談（C1）對舊表幾乎沒差（discordtune 4/0）；較大的表（C2）超過記憶體停止條件（+43 MiB）且傷到保守檔（1/11，$p = 0.006$）。重排器三種配方 × 三個種子在 chatclause 上以 $U_2 =$ 改對 $- 2 \times$ 改壞 比：原配方重跑（R0）每個種子都輸出貨的 SJ-SR-6-14M（U_2 -34、-38、-45），加全部新文字的配方（R1）贏（+24、+22、+11），新文字一半（R$\frac{1}{2}$）+30、+8、+4；R1 中位數的種子（5,211→5,275，106/42，$p < 0.0001$）成為 會意 SJ-SR-7-15M。它在新解碼器上自己的貢獻（CAND 對 B6+SR6，聊天標準）：discordtune 4,558→4,620（88/26）、錯字回報聊天 24/1、書面 18/3；wikitune-clean 書面輸 1,634→1,616（25/43，$p = 0.039$），我們把它讀成口語訓練文字變多的代價（推論；這個歸因沒有量過）。最後的六單位判讀（三檔 × 兩種設定）每個單位都過第 1、3、4 條與詞包檢查，三檔都不過第 2 條的 dev302 書面（15/9、18/12、15/9；已知例外），保守檔不過打字測驗書面（4/3，格 $(1, 1, 4)$ 幾乎不重排，三列是解碼器的錯）。基準測試上 v0.6.0 把錯字回報 80 列聊天從 33 帶到 70、書面從 29 帶到 61，discordtune 聊天 87.90%→93.69%，lytune 聊天 $+295/-66$，保留集口語半邊 115→124（$p = 0.012$）；新聞半邊 125→128（$p = 0.25$）。Linux CLI 的記憶體聊天 271→282 MiB、書面 376→380 MiB（§rl-061–§rl-068）。各階段與定案判讀用的是調參層、開發層、回歸層的集合與檢查集，不是沒看過的判讀集；公開的紀錄裡沒有這一代模型的單次確認（附錄 E）。

解碼器裡的讀音項（R1）。 第一個離線設計（R1，重排前在解碼器分數上加讀音項，12 格）在 dev302 書面（0/1）與錯字回報（2/3）各差 1 列；兩個零成本檢查找到「一」的變調－ˋ被扣 1.39、「長」= ㄓㄤˇ的底分。R1b（變調免扣）與 R1c（551 個底分讀音改用多字詞估計）四條全過；R1c 的 B-f 臂修好錯字回報的「長歪了」、其他逐列相同。產品設計之後重做（R1d），以遵守「只有語意校正分級」的規則：讀音項放進兩種設定、語意校正關也生效的解碼器路徑分數——R1d 在聊天語意校正開時不過（discordtune 標準 60/51、cvtune-clean 15/38）。R1d 同時把常談移出融合，我們把退步至少一部分歸給這件事（三件事一起換，沒有拆開量）；只搬讀音項的 R1e 八種設定全過（聊天 discordtune：關 17/0、保守 16/0、標準 7/1、積極 6/1；書面 cvtune-clean 14/0、12/1；dev302 全部 0/0）。在 v0.6.1 底層重新量測（R1f），第 4 條（錯字回報；R1 的登記裡編成第 3 條）差一句「搞完這波」兩次（0/2）：詞庫的「波」ㄆㄛ（-4.02）高於辭典正讀ㄅㄛ（-4.72）。零成本檢查找到 12 個這類字；R1g 把它們免扣，同一條變成 0/0（淨值要 > 0），所以也「照登記不過」。R1g 以記錄在案的例外往下走：唯一沒過的那一格是 0/0、其他全過，R1g 照出貨規則進兩份沒看過的集合一發確認，確認片在任何數字之前鎖定（§rl-086、§rl-087）：wikijudge5 書面標準 $+15/-0$（$p = 6.1 \times 10^{-5}$）、CE3 cjudge-1 聊天標準 3/4、90% 下界 -0.089 個百分點（門檻 ≥ -1）；兩條都過。出貨的實作對 v0.6.1 在 dev302 與 typing76 逐列相同、錯字回報聊天 $+6/-0$（$p = 0.031$），閒置窗多 ≤ 7.5 MiB、p95 量不出差。fresh verifier 在契約自己造的一句上量到一個顯示層的退步：讀音項換了錨點候選，融合之後就不再選對的字串，雖然兩個候選的分數都沒變（照實記下、沒修；§rl-081–§rl-085、§rl-088、§rl-089）。

重排器接續訓練與新融合格（SJ-SR-8-15M）。 兩列含「權限」的錯字回報錯在重排器的訓練文字被 `TWPhrases` 轉過（權限 → 許可權）。不套表從頭重訓（SR-TW）三個種子都輸出貨的重排器（chatclause U_2 -52、-44、-46），第 1、4 條不過。同種子成對比較（SR-TW-P）顯示輸的幅度不超過重訓的流動：任兩份重排器在 chatclause 互換約 90–100 列、discordtune 65–80 列，淨值在 ±12 內（$p > 0.2$）；132+106 個改壞列裡沒有一列含 `TWPhrases` 鍵，改對列裡有 26+7 列（權限 12、字體 4……）。從出貨權重接續訓練（SR-TW-W：3,000 步、1e-4，W1）把流動壓到 21–33 列，但第 1 條的比例仍不過（$12/9 = 0.75$）。修常談的句首估計（SR-TW-C，再到溫和版 SR-TW-M）讓四列「權限」都對，W1 ＋溫和版只差第 1 條（13/11，比例 0.85）。只用含目標詞的段落針對性訓練（SR-TW-T）修好「權限」卻傷到名單檢查集（書面 1/5）、流動變多（38–54 列）。幾種接續配方與種子（SR-TW-V，11 臂）沒有把流動壓到種子雜訊底線以下（W1 對自己的兩個新種子互換 39 與 33 列；門檻 26.5），只有 V4-s3 過第 1–4 條（16/9，比例 0.56）。它在 wikijudge7 確認不變

差（書面標準 2,255→2,254，5/6，下界 −0.25 個百分點），TW0 調參集也重建成不套 `TWPhrases` 的版本給第 3 條用。全盤清查（SR-TW-F）發現在舊書面格 $(0.5, 2, 0.5)$ 下出貨與 V4-s3 幾乎一樣（4,182 列裡 3,356 對 3,349），換強一點的格兩者增幅也差不多（$(0.8, 2, 0.25)$ 時 3,420 對 3,424）：書面的進步幾乎全是融合格、不是權重。兩次確認在新集合上定下格：FUSE-1 在 wikijudge8（2,844 列），書面 $(0.8, 2, 0)$ 對 $(0.5, 2, 0.5)$：90/45，淨值 $+45$，$p = 0.0001$，比例 0.50；聊天 $(1.1, 2, 0)$ 對 $(0.7, 2, 0)$：43/17，淨值 $+26$，$p = 0.0011$，比例 0.40；FUSE-2 在 wikijudge9（2,827 列），聊天保守 $(1.3, 2, 1)$ 對 $(0.7, 2, 1)$：68/19，淨值 $+49$，下界 $+1.20$ 個百分點；書面保守 $(0.8, 2, 1.5)$ 對 $(0.5, 2, 1)$：14/0，下界 $+0.28$ 個百分點。聊天的格是在聊天文字（cvtune-clean-tw0 與 discordtune）上選的，但聊天判讀集 cjudge-1 已被 R1g 用掉，所以在維基文字上以聊天設定確認；它們能不能轉移到聊天只靠下面的第 1 條。產品（SJ-SR-8-15M、新格、常談溫和版）對 v0.6.1 三檔四條都過：標準 discordtune $+29/-11$（淨值 $+18$，$p = 0.0064$）、cvtune-clean-tw0 下界 $+0.10$ 個百分點、錯字回報 $+24/-6$、dev302 書面 $+9/-2$、typing76 不變；保守 discordtune 淨值 $+35$（$p < 0.0001$）、錯字回報 $+19/-0$；積極淨值 $+9$（$p = 0.12$）、$+23/-6$。第 1、3 條的 12 個 GCP 執行在第二台機器逐位元組相同。兩個已知限制接受、不重選格，因為重選會讓 FUSE-1 的確認作廢：錯字回報第 179 列（「權限加好了」）書面標準與積極當時仍錯；164 列的改選學習檢查裡，書面設定教過一次之後比 v0.6.1 少學會 5 列（140 對 145）。紀錄當時的推論是學習項和 n-gram 同一個尺度，書面 α 從 0.5 變 0.8 之後它的份量相對變小；同一份紀錄 2026-11-10 的更正指出，這幾組的教學列第一次就打對、當時還沒有學習紀錄，而且學習項放大到 2.5 倍也只多對 1 列，所以原因還沒有定論（§rl-098–§rl-115）。接著 v0.7.0 在遮罩裡加了重排保護詞表（第 3.5 節），重排模型不能換掉「權限」與另外四個常用詞 [50, 51]。

句首單字的改選學習（SINIT）。 三個登記過的學習規則版本，各在專案自寫的 teach ／ rare ／ common 案例檔上量，再在新的 6 組確認檔上量。修訂七：案例檔語意類淨值 $+19$，但確認把 3 列「common」由對變錯（上限 1）：不過。修訂八：案例檔 common 翻 0；確認翻 2（都是「帶我出去玩」，和教的「代我出席會議」前三個音節相同）：不過。修訂九（三層前綴鍵）：案例檔 rare 24 對 12、common 翻 0；第三份確認檔 13 對 7、common 翻 1（上限 1）、淨值 $+5$：通過，兩台機器逐位元組相同。沒有學習時，每一版八個集合都和基準逐位元組相同（§rl-090–§rl-094）。

其他有量到選字效果的出貨片。 輕聲容錯（輕聲詞打成本調）讓 300 詞檢查集聊天 177→277、書面 161→264，改壞一列錯字回報（接受）。讓詞包詞保留詞包分數（pack-oov）在解碼器層修好 51 組名字，其中 8 組仍被重排器翻掉——只因為用出貨旗標重跑檢查才抓到。一開始約 10,700 個詞、8 類的 Wikidata 名稱詞包（台灣人物、電視劇、遊戲、樂團、虛構角色、動畫影集與電影、漫畫）沒有改變任何 discordtune 列、調參集 1/0、4/0，只開動漫三類時完全沒變；它做成選用的分類詞包，不進預設詞庫。之後 44,552 詞的動漫詞包預設開。模糊音輸入（四組加鄰鍵容錯）預設關出貨：它的扣分 $\eta_{\text{sound}} = 2.5$ 是四格調參集乾淨輸入改壞率都 $\leq 0.5\%$ 的最小值，但在 dev302 聊天改壞 2 列，兩列都是重排器而不是解碼器造成的（附錄 D.4；§plan-010、§me-020、§rl-013、§rl-056）。

D 負面結果的完整清單

這份附錄列出每一條寫成「照登記不過」、作廢、撤回或停下的線，數字照紀錄。表 6 把主要的幾項分了類。

D.1 判斷器與大型語言模型

- **N0，本機 LLM 重排。**用 Gemma 4 E2B 從 unigram 候選中挑（M1）在開發集達 91.4%，RTX 3070 上 p95 848 ms；門檻是 95% 且 $\leq$ 300 ms。讀音約束的字級生成（M2）更差，

82.5%，偏好流暢的常用詞（拭鏡布變試鏡布）。上限是候選涵蓋：前 16 名含正解的只有 94.7%（§rl-001）。

- **N1，雲端判斷器**。32 組供應商／推理設定，最佳 92.7%（p95 3,146 ms），1 秒內最佳 92.4%（p95 740 ms），全部卡在候選涵蓋。推理關閉多半不損準確度（§rl-002、§plan-006）。
- **S5j，裝置端基礎模型當判斷器**。Apple 裝置端模型從 8 個候選挑一句，discordtune 從 83.5% 掉到 75.4%；換候選順序就改選 49–55%；加前文更差。同一個實驗在 dev302 是 +27 列（$p = 0.0003$），這就是 dev302 只當挑戰集、不當關卡的原因（§plan-013、§me-003）。
- **S5k，本機評分器**。沒有一個本機模型過預先寫死的門檻；最好的 Qwen3-1.7B 4-bit 樣本外 87.2% 對 84.4%（$p = 0.081$），逐候選評分每列 p95 510–640 ms，門檻 300 ms（§plan-021）。

D.2 模型與訓練目標

- **S7b，更大的重排器**。L（3,020 萬參數）在公開調參集更好，在作者的聊天上 −30 列、輸 M −97；XL 放不進 8 GB GPU。去掉和訓練文字重疊的調參列（cvtune 507、wikitune 671）不改選到的值與結論（§plan-014、§rl-044）。
- **S7e，「改壞 1 句 ≈ 改對 10 句」**。在這個權重下 M 在作者的聊天是改對 70、改壞 10，加權效用 $U_{10} =$ 改對 $- 10 \times$ 改壞 $= -30$，L、XL 的 U_{10} 是 −138、−54；v0.4.0 的語意校正預設關。事後算天花板才知道這個權重不能用：就算直接在判讀集上挑值，最好的淨值也只有 4,091 列裡的 +17 到 +23，和只換平台與種子的雜訊同一量級（兩次其他都相同的訓練差 32 列；S7f 用的雜訊界線是 −21）。權重換成 1:1 計分加改壞率上限（§plan-014、§me-011）。
- **S7f，用真人口語重訓**。A1 加兩千萬字開放授權的口語文字；A2 再加兩家開源模型各 250 萬字的合成句。1:10 權重下 U_{10} 是 A1 −126、A2 −107（雜訊界線 −21）；資料清理的錯誤修正之前是 −46 與 −171，結論相同。2026-10-18 用 1:1 重新評分（E-A），A2 在 chatjudge-2 的估計淨值 +16，低於最小可偵測效果 +32，開發層改壞率上界也超過 0.5%；每臂三個種子後估值仍低於 MDE。這些口語文字仍進了 SJ-LM-7 與 SJ-SR-7，在那裡確實有幫助（§plan-014、§ip-004、§ip-005）。
- **rerank-size，三元解碼器上的 L 與 XL**。L188（書面）五格都超過 0.5% 上界（最低 0.574%）；XL 選到的格書面 0.430%、聊天 0.400%；登記的 Holm 檢定在看到的效果量下過不了；之後在現行標準下重開，還沒重跑（§rl-045、§rl-052）。
- **L0，M 上的 listwise 目標**。單獨挑對率掉到 19.1%，discordtune 對出貨 M 淨值 −38；兩條門檻都沒過（§ip-006）。
- **P1 ／ P2 與蒸餾線**。0c 在現行候選上重新量測，對 discordtune 淨值 +105，但開發層上界 0.61%（$> 0.5\%$）；蒸餾進 M 規模的學生（P2）−2（上界 0.39%）；學生加讀音輸入（B）−2；拿掉維基列加零訓練診斷（C）−35；L 規模學生 +7；位置級蒸餾一致率 55.3% 對門檻 72%、−41。學生線照暫停條款結案（§ip-006）。
- **P3 與長前文系列**。學生前文從 2 字加長到 63 字：−39。出貨 M 不重訓直接餵 64 字：discordtune −1、tune −7，不顯著。CTX64 系列（重訓、接續、接續加 rehearsal）之後過不了「三個種子 U_2 都 > 0」的方向判準；量尺檢查 CTX64-M 顯示出貨格是替一個種子選的：同配方換種子在選值集上對它 U_2 −50、−63，所以這條判準分不出重訓的模型和換一個種子；在分半的另一邊各自選格後，挑戰者 U_2 0、−9、−32，對照種子雜訊 −6、−17——沒有方向。前文欄保留結尾標點反而變差（U_2 −26、−18、−17）。這條線暫停（§ip-006、§rl-080、§me-023）。

- **SR-TW 從頭重訓與 SR-TW-T**。見附錄 C：從頭重訓輸的幅度不超過同配方重訓的流動；針對性訓練修好目標列、名字檢查集變差。
- **E-B，讀音前綴輸入格式**。一個種子的試點：調參層淨值 +32 對對照臂 +91；停在試點（§ip-005）。
- **E-F，學習式閘門**。用數字特徵的代價敏感分類器取代 (α, τ, δ)，比同規則多淨值 13，但贏不過出貨值，上界超過 0.5%（§ip-005）。

D.3 解碼器項與資料

- **針對混淆字的合成句**（第 2 輪，2026-10-03）：21,611 句針對混淆對生成的句子讓調參集和聊天集都退步（§rl-006）。
- **先驗混入語料頻率**（混合權重 0.3 或 0.5；和 Kneser–Ney 的 β_p 無關）讓聊天少對 100 多句：語料以維基為主，先驗吸進了書面語的偏好（§rl-011、§rl-012）。
- **維基台灣正體轉換（W、W2)**。守門沒否決；保留集聊天 −2、書面 +1；使用者回報「的市佔」變成「的士站」；不出貨（§plan-020）。
- **P1a，在／再的詞態分類器**。只訓ㄗㄞˋ就觸發停止規則：cvtune 聊天「再 > 在」11→10（要求減半）；分類器的字位正確率低於 n-gram；取樣上限造成類別先驗扭曲，融合在審查前混用了自然對數與 log10（§plan-011、§me-010）。
- **s4-bayes，學習的 Dirichlet 後驗**。40 組參數沒有一組符合條件；鏡像學會率每組都低於現行規則（0–3/8 對 7/8；§plan-022）。
- **derisk-1015，E1 三元表大小**。門檻 15、3、2 與不剪的三元表裡，只有不剪的那一檔（71,858,873 條，1,133.3 MB）在它選定的值上過登記的判讀；在出貨的 λ 重判沒有一檔過（$\alpha = 0.0083$），我們把增益讀成隨剪枝單調變小（從六種表大小推論）。770 萬條的表仍在另一次以出貨 λ、關 Kneser–Ney 的重判之後出貨（見上）。無損壓縮能省 bigram 7.8 MB 的安裝大小；4 位元量化顯著變差（§rl-020、§rl-024、§rl-027）。
- **帶 Kneser–Ney 重調** $\lambda \times \beta$：調參集把 λ 推到格點上緣，discordtune 顯著變差（聊天 −39、書面 −43，$p \le 0.0001$；§rl-022）。
- **E4(b)，數字／英文之後的前文**：主判讀 +18（$p = 0.0198$），但 wikitune 2/12（$p = 0.013$）與 2/11；守門不過。**E4(a)**，句中標點之後不重設前文：−1 與 −8。**E3**，整串只打聲母：top-1 比例 0.789/0.808 對門檻 0.9，瓶頸在排序（oracle@64 仍有 289/302），每列 148 ms（§rl-023、§rl-025、§rl-026）。
- **讀音感知的計數（B)**：bigram 分讀音計數讓 lytune 聊天少對 1,457 列（958 列是嘛 → 麻、493 列啦 → 拉）：規則標註在抽樣的 157 個位置錯 64 個（40.8%；副詞「地」標成ㄉㄧˋ、助動詞「得」標成輕聲），停止條件 10%。**A2**，語尾助詞一聲讀音降權：lytune $\delta = 0.25/0.5$ 時 −213 ／ −508。A1，44 字輕聲降權表，沒改壞任何一列，出貨（§rl-038）。
- **E-D，候選涵蓋**。每個讀音的候選從 12 放寬到 24 ／ 48 ／ 1000，聊天 oracle@64 最多升 0.03 個百分點。**E-C**，解碼權重與融合聯合調：聊天最好的格比出貨點多 5 列；書面最好的就是出貨點（§ip-005）。
- **BRIDGE-1 ／ 2，隔一詞的詞類橋接項**。聊天調參格都是正的（cvtune +8 到 +17），書面已有三元表的地方是負的，改壞 80 對改對 107；只用在聊天、在這條線沒看過的過濾後

chatclause 上（BRIDGE-2）：0/2，而且登記的臂不是先前量過的那一臂（記錄在案的登記錯誤；§rl-070、§rl-077）。

- **LEXGAP-0 ／ 1，後綴合成詞（X 氏、X 室）**。疊加層加 694 詞，十個集合／設定組合一列都沒改到，因為疊加層詞拿底分再扣 1.0；把它們放進計數詞表重建 bigram（691 詞，+2,672 條二元組）只修好一列（桑拿室）。這條線結案，詞表進下一次重建（§rl-095、§rl-096）。

- **Moore–Lewis 挑選** 論壇 A 與 g0v 文字（見上）：比全收差（§rl-061）。

- **HPLT 與維基文庫當長前文評測文字**。HPLT 3.0 繁體論壇／部落格文字三次各抽 300 列，12.7%、5.3%、6.7% 的列不能用，主要是 SEO 插進去的真詞；維基文庫的公有領域散文 37% 是簡體原文，濾掉之後第二次校對整體仍要改 13.8%；兩份都放棄（§me-016）。

- **OpenCC 轉換是根因**。`TWPhrases` 一直在改寫 127 個詞庫詞（71 個常用：支持、設備、程序、權限……），其中 47 個有同讀音同長度的對手；94 句構造句上，出貨重排器在解碼器對的列裡改壞 7/49（聊天）與 7/46（書面），重訓的 W1 改壞 1 與 0。另外，`STCharacters` 把「湿」只轉成「溼」而 MediaWiki 用「濕」；前 2,000 篇裡「溼」有 381 次來自轉換後的簡體句、18 次來自繁體句（「濕」是 29 與 135），而論壇 A 用量是濕 1,418 對溼 106。維基一半的句子（49.3%）真的是簡體原文；判成簡體的句子裡 20,436 句沒有任何簡體專用字。一個轉換修正片（TW-CONV）用 LLM 輔助逐句校對加盲抽查建了 29,918 列的校對表（gap1：該改 69 處修對 64、該留 41 處改壞 0），下一次重建會用（§rl-107、§rl-117、§rl-118）。

D.4 融合、決策與不成立的確認

- **rerank-formal 標準格**：主判讀過、改壞率守門不過（調參資料上 1.72% 對 0.50%）——就是讓守門改成上界的那一次（§rl-039、§rl-040）。

- **P4b、P4c**：見上；P4c 在 chatclause 選的格在選值集上比例 0.45、到 discordtune 0.60，和 P4b、P4d 同樣的上移（0.33→0.58；§rl-050、§me-017）。

- **DECIDE-0 ／ 1，放寬書面融合**。探索裡八個放寬的書面格都過方向規則，最好的 $\alpha = 0.75, \delta = 0$（三個集合淨值 +108，比例 0.42）；在沒看過的 wikijudge4（2,820 列）確認 $+67/-48$、淨值 +19、$p = 0.0928$、比例 0.72：三條裡兩條不成立。之後 FUSE-1 用新權重確認 $(0.8, 2, 0)$ 在 wikijudge8 通過（比例 0.50）（§rl-075、§rl-076）。

- **模糊音扣分** η_{sound}（見上）：dev302 聊天 0/2；兩列都是解碼器已經把容錯詞壓下去之後、在重排器輸掉的。

- **R1、R1d、R1f、R1g**（附錄 C）：三次差一列、一次 0/0，然後才例外；R1g 之後確認成立。

- **SINIT 修訂七、八**：common 列翻 3 與 2，上限 1。

- **SR-TW、-W、-C、-M、-T、-V**：最後一輪探索 11 臂，沒有一臂全過；V4-s3 是唯一過第 1–4 條的。

- **LOSS-0 ／ L5-0 診斷**（探索）：整條管線後仍錯的同音錯裡，28–64% 是重排器偏向正解但閘門留住解碼器的列（第 4 層），34–52% 是重排器也偏錯的列（第 5 層）；書面第 4 層有三到四成被 δ 擋下、幾乎沒有被 τ 擋下的。更強的重排器在兩個聊天調參集能救的上限是 246 列，但抽樣的列裡約一半判成無法判斷，真正能救的可能只有一半（推論）。這一片的第三個檢查沒過自己登記的分數比對容忍度（$\log_{10}$ 差 $\le 10^{-6}$；實際最大差 1.6×10^{-5}），表只照錄、不判讀（§rl-073、§rl-074、§rl-078、§rl-079）。

D.5 退回或延後的產品改動

- 依按鍵順序把不可能是注音的鍵序當英文，預設開出貨，同一天因快打誤觸太多（一個音節後面的鍵先到）退回；這條規則之後帶著「鍵序顛倒不當英文」的守門回來，兩個相關開關從 v0.7.0 起預設關（§me-022；[50]）。
- 「改選後其他字不動」在第五次審查量到「改選後繼續打字」的流程變差（整句正確 1500→1438，共 2165 句）後暫停；之後重新接回，在 v0.5.0 出貨（§plan-019；[50]）。
- 貝氏學習規則、全域學習層與縮寫模式仍關著。

E 工程上的控制與流程偏差

E.1 工程上的控制

同一次比較的所有臂在同一種 CPU 上跑：同一份 135 列在 Intel 桌機與 AMD EPYC 9B14 雲端 VM 上選字相同，小模型分數每列差到 7.1×10^{-6}（§me-015）。2026-10-29 起，確認與出貨標準的量測在兩台機器各跑一次、逐列結果逐位元組比對（小模型分數差在 10^{-5} 量級、在容忍度內比）；保留集只由一個 agent 量一次。每次跑之前核對每個模型、詞庫、詞包與評測檔的雜湊，CLI 印出的設定標記要等於預期值。程式審查與驗收由獨立的 agent 做。保留集只由這種 agent、只在推 tag 前量，只回摘要行。

E.2 流程偏差

流程在下面這些地方違反了自己的規則，或沒有照規則做；列出來是因為它們限制了結果能主張的範圍。(1) 出貨標準看過數字之後放寬過兩次（2026-10-20）；只在放寬後過關的片在 chatjudge-2 重新確認（§rl-049、§rl-053）。(2) P5（讀音通道）在顧問看過兩個回歸集之後、沒有在沒看過的集合確認就出貨；當時沒有沒用過的聊天判讀集（chatjudge-2 已被 P4d 用掉，判讀集每月最多抽一份），等十一月抽 chatjudge-3 再確認的提議沒有採用，P5 和常談一起在 v0.5.0 出貨、不等確認（§rl-054、§me-018）。(3) R1g 在第 4 條 0/0 之後例外進確認（§rl-086）。(4) P4d 的確認寫成 README 的一節，不是帶 MDE 的一頁登記（§me-017）。(5) DECIDE-1 的第二份來源在 Mac 上跑，違反「Mac 不跑重運算」（§rl-076）。(6) 三元片 B2 的停止條件（手動詞 2/15 錯，上限 1）在 B3 跑完之後才報；那兩列之後記成已知問題、B2 的條件放寬、片繼續（§rl-034、§rl-036）。(7) model-v5 的保留集在含它的版本發佈之後才跑（§me-009）。(8) L5-0 檢查 3 程式裡的容忍度和登記不一致（§rl-079）。(9) BRIDGE-2 登記的臂不是 BRIDGE-1 量過的臂（§rl-077）。(10) 幾個早期由實作 agent 寫的「結案」在 2026-10-20 重新歸類為「照登記不過」，因為當天定下的規則是登記沒過不算結案（§rl-052）。(11) v0.6.0 那一代模型是表 2 裡最大的一步（用論壇與聊天頻道文字重建的 bigram，跟著它建的詞類、Kneser–Ney、三元表與常談檔，以及重排模型 SJ-SR-7-15M）；它照出貨標準的定案判讀出貨，用的是調參層、開發層與回歸層的集合，這些集合在選臂與選值時都看過，公開的紀錄裡沒有它的單次確認（§rl-059–§rl-068；[50]）。在沒有用來選值的資料上，本文對它有的是監測用的保留集（表 3）。(12) 表 1 列出跑過的每一次單次確認，所以其他不在表裡的出貨改動也沒有單次確認，例如：v0.7.0 的口語項句首估法（原因見第 3 節開頭）、同分讀音的規則與重排保護詞表；v0.5.0 的兩條改選與學習規則（改選後其他字不動、逐字改選學成新詞；§plan-019；附錄 A.8；[50]）；組字行為（v0.6.0 加長的組字區；[50, 51]）；以及詞庫各層、詞包與輸入容錯（附錄 A 與附錄 C 最後一段），紀錄裡有數字的，量的是檢查集與已經看過的集合。

F 專案第一天對既有輸入法的快照

專案第一天，用送鍵的自動化工具（§me-005、§me-006）把同一批句子打進三套既有輸入法：macOS 內建注音、小麥注音（MIT，片語源自 libtabe）[37] 與一套商業輸入法。23 句書面句子三

套各錯 3、10、8 句；53 句口語句子 8、19、22 句；845 個私人聊天子句 100、199、237 句；294 句政府新聞稿 64、85、94 句。內建注音每一輪都最好，而且被發現會在測驗中學習：它學到的 2,647 筆詞組裡，847 筆出現在它自己的測驗輸出（§rl-004）。

這些數字是單獨一天（2026-10-03）的快照，有四個限制。這些產品持續在改版，快照沒有重做。內建注音帶著它的學習狀態。讀音來自專案讀音工具的早期版本，一個字標錯，三套一起錯。集合也是專案自己的：76 句書面與口語句就是表 2 的回歸集 typing76，善解之後在它上面調過；294 句新聞稿 2026-10-29 起成為訓練語料、不再是測試集。這份快照是專案的起因，不是和善解的比較，旁邊也不放善解的數字。

開發紀錄裡另外看過的工具：唯音（vChewing）的中英混打 [53]；閉源的晶晶輸入法（ZingIME；小麥分支加本機微調的 Qwen2.5-0.5B）；Python 研究核心 zaoseq-bopomofo（丟掉已知詞裡沒出現過的多音字讀音，並報「改錯 $\leq 0.5\times$ 改對」的通過條件）；以及 Jevboard（Windows 上請 Jev 判斷模型挑字的小工具）（§rl-057、§plan-015、§plan-016）。

G 全域學習層掃描

表 10: 詞類項開著時的全域學習加分 ε（2026-10-08）：鏡像學會率與污染列。沒有詞類項時，$\varepsilon = 0.5$ 是學會 8/9、污染 0。

$\varepsilon_{\text{global}}$	學會	污染列
0	0/8	0
0.1	3/8	1
0.5	4/8	3
1.0	6/8	4
1.5	7/8	7
2.0	8/8	9
6.0	8/8	29

H 模型檔與代號

v0.6.1 出貨的重排模型（SJ-SR-7-15M）的融合三元組，保守、標準、積極三檔依序是聊天 $(0.7, 2, 1)$、$(0.7, 2, 0)$、$(1.0, 2, 0.1)$，書面 $(0.5, 2, 1)$、$(0.5, 2, 0.5)$、$(0.5, 2, 0.5)$；v0.7.0(SJ-SR-8-15M) 的在第 3.5 節。

表 11: v0.7.0 的模型檔，以及各自的中文功能名與代號。v0.6.1 出貨的是同一組檔，只有重排模型不同，是 SJ-SR-7-15M（規模相同）；SJ-SR-8-15M 於 2026-11-06 在 main 分支取代它。

中文名	代號	檔案	規模
通語	SJ-LM-7-9.7M	bigram.sjlm	9,709,510 條二元組合，396,927 詞
辨類	SJ-WC-6-512C	classes.sjc	512 類
承接	SJ-KN-2-397K	kn.sjkn	396,927 詞
連文	SJ-CX-2-7.7M	trigram.sjt3	7,700,964 條三元組合（書面）
連文	SJ-CX-2-950K	trigram-chat.sjt3	950,000 條三元組合（聊天）
會意	SJ-SR-8-15M	tinylm.sjtl	14,646,912 個參數
常談	SJ-CL-2-69K	colloquial.sjlm	69,457 詞